



Review article

WebAssembly and security: A review

Gaetano Perrone^{ID}*, Simon Pietro Romano

University of Naples Federico II, Department of Electrical Engineering and Information Technology, Via Claudio 21, Naples, 80125, Italy

ARTICLE INFO

Keywords:

Wasm security
WebAssembly security
Security review
WebAssembly review
Cloud computing security

ABSTRACT

WebAssembly is revolutionizing the approach to developing modern applications. Although this technology was born to create portable and performant modules in web browsers, currently, its capabilities are extensively exploited in multiple and heterogeneous use-case scenarios. With the extensive effort of the community, new toolkits make the use of this technology more suitable for real-world applications. In this context, it is crucial to study the liaisons between the WebAssembly ecosystem and software security. Indeed, WebAssembly can be a medium for improving the security of a system, but it can also be exploited to evade detection systems or for performing crypto-mining activities. In addition, programs developed in low-level languages such as C can be compiled in WebAssembly binaries, and it is interesting to evaluate the security impacts of executing programs vulnerable to attacks against memory in the WebAssembly sandboxed environment. Also, WebAssembly has been designed to provide a secure and isolated environment, but such capabilities should be assessed in order to analyze their weaknesses and propose new mechanisms for addressing them. Although some research works have provided surveys of the most relevant solutions aimed at discovering WebAssembly vulnerabilities or detecting attacks, at the time of writing there is no comprehensive review of security-related literature in the WebAssembly ecosystem. We aim to fill this gap by proposing a comprehensive review of research works dealing with security in WebAssembly. We analyze 147 papers by identifying seven different security categories.

We hope that our work will provide insights into the complex landscape of WebAssembly and guide researchers, developers, and security professionals towards novel avenues in the realm of the WebAssembly ecosystem.

Contents

1.	Introduction	2
2.	Preliminary concepts.....	2
2.1.	WebAssembly and industry	2
2.2.	WebAssembly introduction	3
2.2.1.	WebAssembly components	3
2.2.2.	WebAssembly memory layout and binary format	3
2.2.3.	WebAssembly use-cases.....	3
2.3.	Low-level vulnerabilities in software.....	4
2.3.1.	Security approaches to discover software vulnerabilities	4
2.4.	Smart contracts	4
2.4.1.	Vulnerabilities in smart contracts.....	5
2.5.	Trusted execution environments and security enclaves.....	5
3.	Literature review process.....	5
3.1.	Automatic search.....	5
3.2.	Works inclusion and exclusion criteria	6
3.3.	Works classification	6
4.	Related works.....	6
4.1.	Security studies.....	6
5.	WebAssembly and security: a review	7

* Corresponding author.

E-mail address: gaetano.perrone@unina.it (G. Perrone).

5.1.	Security analysis	7
5.2.	Empirical studies	8
5.3.	Use cases in cybersecurity	8
5.3.1.	Browser extensions	8
5.3.2.	Cryptography, trusted execution environments and privacy	8
5.3.3.	Identity and access management	8
5.3.4.	Cloud, cloud-edge computing and IoT	9
5.3.5.	Face recognition systems	9
5.4.	Attack scenarios	9
5.4.1.	Side-channel attacks	9
5.4.2.	Malware detection evasion	9
5.5.	Attack detection solutions	9
5.6.	Security enhancements	9
5.6.1.	Works that enhance WebAssembly indirectly	11
5.7.	Vulnerability discovery	11
5.8.	Other works	11
5.8.1.	Preliminary works	11
5.8.2.	Works about vulnerability discovery approaches	11
5.8.3.	Comparison with other works	13
6.	Discussion	13
6.1.	WebAssembly security	13
6.2.	Detect attacks and discover vulnerabilities in WebAssembly	13
6.3.	The interest of WebAssembly for cybersecurity use-cases	13
6.4.	Enhance WebAssembly security	13
6.5.	Results availability	15
6.6.	Research gaps in WebAssembly and security	15
7.	Conclusions	16
	Declaration of competing interest	17
	Appendix. Publicly available source code repositories	17
	Appendix. Data availability	17
	References	17

1. Introduction

WebAssembly is a formal specification for portable machine code, born to allow the realization of portable code developed in any language and executed by modern browsers. This technology was developed to provide a client-side solution for browsers, but several cloud providers are nowadays offering WebAssembly runtime environments for the server side. This reveals the key innovations that WebAssembly promotes in terms of performance, isolation, distribution, and modularity. With the spread diffusion of WebAssembly, it becomes crucial to research the liaison between this technology and security. Security research can involve different topics, such as empirical studies for evaluating the presence of security flaws in WebAssembly design and systems, the realization of solutions aimed at enhancing WebAssembly security, the implementation of novel approaches for discovering vulnerabilities or detecting attacks in WebAssembly, or the proposal of WebAssembly solutions for security-related use cases. Although several studies have been developed in the above-mentioned research fields, to the best of our knowledge there is no work that provides an exhaustive survey of the existing literature.

This work aims to contribute to this direction by providing a comprehensive review of the most relevant works that have investigated the liaisons between security and the WebAssembly ecosystem. We analyze 147 works, classifying 116 into seven security categories and describing the remaining 31 additional works that, while not fitting into a specific security category, provide fundamental knowledge for expanding the literature on WebAssembly and security. We hence provide a security overview of the WebAssembly world, which will hopefully represent a starting point for researchers looking to investigate future works, as well as for security practitioners aiming to integrate this appealing technology into their systems.

The remainder of this article is structured in six sections. Section 2 conveys to the readers the preliminary concepts functional to follow subsequent sections. Section 3 illustrates the review process

we followed for retrieving works relevant to our review. Section 4 shows existing studies that reviewed WebAssembly research works, with particular emphasis on those that have concentrated their efforts on security, by illustrating the differences with our findings. Section 5 describes the reviewed works, while Section 6 provides a discussion on these by summarizing relevant information and research gaps that could be addressed in future investigations. Section 7 concludes our research path by proposing insights for researchers who want to explore challenging avenues in the WebAssembly security field.

2. Preliminary concepts

In this section, we first describe the WebAssembly ecosystem by introducing the reader to the basics. Then, we highlight the concepts that can be useful to understand the relevant works that will be illustrated in Section 5, namely, smart contracts and security techniques for addressing low-level vulnerabilities.

2.1. WebAssembly and industry

WebAssembly is facilitating the transformation of industries by enabling high-performance applications that can be run both inside web browsers and on edge devices. Web applications continuously evolve by involving more resource-intensive tasks, such as video editing, gaming, and data visualization. The optimized WebAssembly binary code allows high-performing executions compared to traditional JavaScript applications [1]. The WebAssembly technology has been used to develop innovative applications in the Virtual Reality and blockchain fields [2].

In addition, in the industry field, companies are increasingly extending the use cases portfolio by experimenting with the adoption of WebAssembly in additional domains, such as automotive [3], manufacturing [4], and medical sector [5]. These domains can benefit from the adoption of WebAssembly through innovative cloud-edge infrastructures. The growing interest in such a technology has two consequences from a security perspective. Whilst WebAssembly intrinsically provides

security capabilities in terms of isolation, memory safety, and availability, it remains an emerging technology, and its security aspects have not been comprehensively explored so far. This is a crucial point, as numerous applications utilizing WebAssembly handle sensitive data, and the presence of security flaws might have significant repercussions. Our primary objective is to shed light on these aspects by outlining the most relevant works that have explored WebAssembly security from different viewpoints.

2.2. WebAssembly introduction

In 2015, Brendan Eich announced the creation of WebAssembly, an innovative approach to writing assembly programs for the web [6] aimed to replace `asm.js` [7]. The main reasons behind the realization of an alternative approach to `asm.js` were to simplify the compiler options and to increase performance by natively decoding code much faster than JavaScript [8]. In 2017, the WebAssembly Community Group was created with the mission of providing collaboration on defining a pre-standardization of WebAssembly, and in 2019, WebAssembly became a World Wide Web Consortium (W3C) recommendation [9]. Design goals are focused on:

- **Performance:** the code execution should be as close as possible to native code performance by taking advantage of modern underlying hardware features;
- **Security:** the code should be validated and executed in memory-safe, isolated, and sandboxed environments;
- **Portability:** the code should be independent from language, hardware, and platform.

After the WebAssembly foundation, several companies and non-profit organizations such as Bytecode Alliance [10] were interested in supporting the project.

WebAssembly was designed to build C/C++ projects, but today, there is a strong community effort to increase the number of high-level languages supported. At the time of writing, it is possible to compile Python, Ruby, PHP, C, and Rust programs and generate WebAssembly code that can be executed into a WebAssembly compiler.

As regards the ecosystem, some companies are following the approach of the Docker community, which developed DockerHub, an open search engine and repository for finding ready-to-use Docker images [11]. In particular, the Wasmer team provides a complete WebAssembly framework, constituted by a WebAssembly runtime for running WebAssembly binaries [12], a Software Development Kit (SDK) to embed WebAssembly programs in any programming language [13], a serverless platform for executing programs developed through the SDK [14], and a publicly available and searchable repository where it is possible to find any WebAssembly binary developed by the community [15].

2.2.1. WebAssembly components

The WebAssembly ecosystem consists of several components that enable the execution of programs (at runtime or compiled) written in multiple languages. WebAssembly used to be mainly focused on executing C programs, but currently, it supports the entire set of most popular programming languages. The developers write a program in their own preferred language. Then, the source code can follow two processing steps: (i) it can be compiled by a *WebAssembly compiler* and executed by a runtime engine, or (ii) it can be interpreted, i.e., a *WebAssembly interpreter* at runtime processes the lines of code and executes them. The execution is orchestrated by a *WebAssembly runtime engine*, that abstracts the execution from the underlying system (i.e., the browser, the operating system, etc.) and eventually executes either the WebAssembly binary file (in case of compilation) or the current line of code (in case of interpretation) on the underlying system architecture.

The development of WebAssembly was motivated by the need to provide a low-level assembly-like language for compiling arbitrary

source code and running it in the browser. As WebAssembly shows a high level of portability, it has been extended to include other platforms. The WebAssembly System Interface (WASI) [16] comprises a collection of standardized API specifications aimed at providing secure interfaces for programs, thereby extending the capabilities of WebAssembly. The API facilitates interaction with underlying functionality such as I/O, clocks, filesystem, HTTP driver, and more.

WebAssembly programs are organized into modules, i.e., units of deployment, loading, and compilation, that have definitions for types, functions, tables, memories, and globals. Each module declares imports and exports, and upon instantiation, a *start function* is automatically invoked.

2.2.2. WebAssembly memory layout and binary format

WebAssembly follows the principle of simplicity by defining a simple linear memory structure, i.e., a mutable array of raw bytes. Each memory has a *metadata region* that keeps information about the memory layout, size, and properties of the memory in the WebAssembly module. Each program loads and stores values from/to a linear memory at any byte address.

The computational model of WebAssembly is based on a *stack machine*. The program's instructions are executed in order and manipulate values by popping arguments and pushing results into the stack. Each instruction is encoded as *opcodes* of one byte (multiple bytes for control instructions), followed by immediate arguments if present. There are several instruction types, namely, control, reference, parametric, variable, table, memory, numeric, vector, and expression.

WebAssembly has two concrete representations, which map to a common structure:

- **text format** (*.wat* extension): designed to let developers understand the structure of a WebAssembly module. It can be used for writing code and compiling it in binary format;
- **binary format** (*.wasm* extension): a compact binary instruction format for a stack-based virtual machine. It is the compilation target of higher-level languages.

2.2.3. WebAssembly use-cases

WebAssembly is designed to ensure high flexibility and portability. As an example, a program written in C can be compiled and executed in any architecture, in the browser, on x86 desktop machines, and in arm-embedded microcontrollers. Typical use cases are focused on browser-based applications such as games, peer-to-peer applications, or image recognition, but there are also other application scenarios [17]. Hilbig et al. (2021) [18] sample 100 random WebAssembly binaries used on the web and underline that WebAssembly is used for gaming, text, and media processing, visualization/animation, as well as demonstrations of programming languages. A relevant report is provided by the Cloud Native Computing Foundation (CNCF), that every year summarizes the state of WebAssembly in the world [19]. According to their study:

- The principal use-cases of WebAssembly are web applications (58%), data visualization (35%), Internet of things (32%), artificial intelligence (30%), games (28%), and backend services (27%)¹;
- Developers are interested in performance benefits, exploring new use cases and technology, and sharing code between projects. They confirm performance benefits when WebAssembly is adopted;

¹ The percentages do not indicate the number of use-cases, but rather reflect the proportion of 255 respondents who selected these categories in a multi-select question about the types of applications or services for WebAssembly.

- WebAssembly is used both for realizing new applications and migrating existing applications.

According to the CNCF study, the source code languages used for compiling WebAssembly binaries are primarily JavaScript, C#, C++, and Python. However, other experts [20] show that Rust is quickly becoming one of the dominant source code programming languages in the WebAssembly ecosystem.

In our work, we highlight the most pertinent cybersecurity-focused use cases documented in the literature (see Section 5.3).

2.3. Low-level vulnerabilities in software

WebAssembly is an assembly language typically used to compile basic code developed in low-level languages such as C. Therefore, it is crucial to introduce foundational concepts surrounding low-level vulnerabilities that could impact a program. In order to understand this kind of vulnerabilities, it is essential to offer an overview of program execution in machines.

Low-level languages such as C and C++ can lead to various vulnerabilities stemming from unsafe memory management practices:

- *Stack-based overflow*: this vulnerability occurs when an attacker can overwrite a memory buffer located in the stack, including function return addresses, leading to arbitrary code execution;
- *Heap-based overflow*: this vulnerability is similar to stack overflow, but it occurs in the heap memory section;
- *Integer overflow*: this vulnerability occurs when the result of an arithmetic operation exceeds the memory size allocated for the variable holding it, potentially leading to memory corruption or unintended code execution.

These vulnerabilities often occur when unsafe functions are input-controllable, allowing a malicious user to send input that overwrites memory buffers.

2.3.1. Security approaches to discover software vulnerabilities

Software vulnerability discovery techniques can be classified into two broad categories:

- *Static analysis*: involves source code analysis without executing the program. It is useful to detect vulnerabilities caused by improper input validation or type mismatches, but cannot discover runtime errors or vulnerabilities caused by complex interactions with the code;
- *Dynamic analysis*: these approaches try to find vulnerabilities in programs at runtime by injecting malicious inputs (fuzzing) or by analyzing the code during execution (tracing or symbolic execution).

These approaches can be further classified into several techniques, as illustrated in Table 1.

2.4. Smart contracts

This section provides context information useful to follow all the considerations exposed in Sections 5 and 6. A common use case for WebAssembly is its application for the implementation of smart contracts. This stems from the distributed nature of smart contracts that fits perfectly with the portability and versatility of WebAssembly. Smart contracts are digital contracts stored on a blockchain, which are executed under a specific set of satisfied conditions. They are inherently related to blockchains, i.e., distributed, shared, and immutable ledgers [31].

One of the most widely deployed blockchain platforms is Ethereum [32], which extends the capabilities of the blockchain technology beyond simple financial transactions. It allows developers to create smart contracts as complex applications with self-executing code. These smart contracts get automatically executed if and when specific conditions are

Table 1

Static and dynamic techniques to discover vulnerabilities.

<i>Static techniques</i>	
Control graph flow analysis [21]	Analyze the graphical representation of all the paths that could be traversed during the execution of a program to discover security issues.
Context-sensitive data flow analysis [22]	Define a set of “contexts”, i.e., scenarios in which a function can be called within the program, and analyze the data flow when the context changes.
Code property graph [23]	Realize a graph-based representation of the analyzed program that can be searched for obtaining relevant information and potentially discover vulnerabilities.
State machine representation [24]	A theoretical model based on program states is used to represent the behavior of the program by potentially revealing security flaws.
<i>Dynamic techniques</i>	
Black-box fuzzer [25]	Send random inputs to trigger crashes and detect vulnerabilities. No knowledge about the application’s internal structure.
White-box fuzzing [26]	Perform fuzzing with full knowledge of the internal structure of the application under test.
Grey-box fuzzer [26]	Combine black-box and white-box approaches to leverage both of them.
Symbolic execution [27]	Analyze the program to determine through symbolic variables the inputs that trigger the execution of a specific part of a program.
Concolic execution [28]	Like symbolic execution, but using concrete values, by increasing the path exploration space.
Tracing and control-data flow [29]	Monitor the program’s control flow and data flow and keep track of triggered vulnerabilities during the execution.
Bytecode instrumentation and run-time validation [30]	Modify bytecode at runtime in order to insert additional checks that can be validated to identify security flaws.

met, hence providing transparency, immutability, and trust in transactions. Ethereum also introduced its native cryptocurrency, known as *Ether* (ETH), which is used to reward network participants for performing computations and/or validating transactions. The Ethereum documentation [33] provides several relevant concepts for understanding Blockchain vulnerabilities:

- *Ethereum Account*: a digital identity on the Ethereum blockchain, allowing users to send and receive Ether, as well as interact with smart contracts;
- *Address*: a unique identifier used for receiving tokens, used to identify the Ethereum Account;
- *Block*: the data structure where the transactions or digital actions are stored;
- *Blockchain*: a database of transactions, duplicated or shared on all computers in the network, ensuring that data cannot be altered retroactively [34];
- *Smart contract*: a program that automatically executes agreements on a blockchain, like a self-enforcing digital contract [35];
- *Transaction*: data committed to the Ethereum Blockchain signed by an originating account, targeting a specific address;
- *Contract account*: an account containing code that executes whenever it receives a transaction from another account;
- *Ethereum Virtual Machine (EVM)*: a stack-based virtual machine that executes bytecode;
- *Externally Owned Account (EOA)*: the most common type of Ethereum account, controlled by a person through private keys;
- *Internal transaction*: a transaction sent from a contract account to another contract account or an EOA;

- *Message*: an internal transaction that is never serialized and exists only in the Ethereum execution environment (i.e., a message is like a transaction, except it is produced by a contract and not an external actor);
- *Message call*: the act of passing a message from one account to another;
- *Delegatecall*: a message call that uses data of the calling contract;
- *Receipt*: data returned by an Ethereum client to represent the result of a particular transaction, including a hash of the transaction, its block number, the amount of gas used, and, in case of deployment of a smart contract, the address of the contract.

Earlier blockchain platforms suffered from performance issues. For this reason, researchers have proposed different consensus algorithms, such as Proof-of-Share (PoS) and Delegated Proof-of-Stake (DPoS). One of the most relevant DPoS platforms is EOSIO [36]. The EOSIO smart contract is composed of several concepts:

- *action*, that represents a single operation for communicating between a smart contract and an account;
- *transaction*, that is composed of several actions;
- *apply function*, that dispatches action handlers for validating smart contracts. The function has a ‘code’ parameter that is used to identify the account that authorized the contract;
- *eosio.token contract*, a standard contract that can be used to transfer tokens with the “transfer” functionality;
- *dApp*, a decentralized application that runs on a decentralized network.

2.4.1. Vulnerabilities in smart contracts

Smart contracts are affected by vulnerabilities that are strongly related to the smart contract domain and to the adopted blockchain platform. He et al. (2022) [37] provide a comprehensive survey of attacks and vulnerabilities in EOSIO systems. Table 2 describes the most relevant vulnerabilities affecting smart controls.

Section 5.5 shows the techniques that have been devised to detect this kind of attacks.

2.5. Trusted execution environments and security enclaves

As it will be shown in Section 5.3, several WebAssembly use-cases and enhancements entail an integration with processor security features. A trusted execution environment (TEE) is a hardened memory section of the processor aimed at protecting the confidentiality and integrity of both code and data [38]. One of the most utilized technologies that implement TEEs is the Intel Security Guard Extension (SGX) [39]. SGX defines the “security enclave” concept, i.e., a secure area where sensitive data and code can be executed. The approach offers scalability by ensuring the execution of an entire application or just a single function in the enclave. Developers can use an API to build secure applications based on Intel SGX. Intel also provides a Software Development Kit (SDK) to handle security enclaves in their code.

3. Literature review process

This section illustrates the process used to review the literature on security for WebAssembly. The review has followed a structured process inspired by the methodologies outlined in two relevant works: Webster and Watson (2022) [40] and Kitchemann and Brereton (2013) [41]. The process proceeded through the following steps:

1. *Automatic search*: We performed an automatic search with specific keywords;
2. *Cleaning*: we removed duplicates obtained from the automatic search, and excluded out-of-scope works that did not satisfy the inclusion and exclusion criteria;

Table 2

Most relevant vulnerabilities in smart contracts.

Vulnerability	Description
Fake EOS Transfer	An improper apply function implementation does not verify the code parameter properly, hence allowing the execution of code logic under fake EOS tokens.
Fake EOS Receipt	If the notification of a smart contract allows unsecured relays, an attacker may invoke a transfer in <code>eosio.token</code> , notify an accomplice who relays the notification to the victim that will pass EOS checks by tricking the victim into providing services to the attacker, such as transferring tokens.
Fake EOS Notice	A variant of Fake EOS Transfer that exploits other parameters even if the system provides a valid apply implementation.
Insecure Message Call	A message call where the authorization is not properly checked.
Greedy Smart Contract	A greedy smart contract can receive ethers, but it contains no functions to send ethers out.
Dangerous Delegatecall	Attacker manipulates <code>msg.data</code> by changing the function called by the victim's contract.
Block Information Dependency	The improper utilization of a block timestamp or block number for determining a critical operation. As these variables can be manipulated by miners, they cannot be considered as reliable sources.
Mishandled Exceptions	Improper exception handling can cause inconsistencies during a transaction.
Reentrancy Vulnerability	Invoke in a reentrant manner a function that was not designed to be reentrant. ^a
Rollback	The attacker exploits an unsafe rollback operation on a blockchain system by conditionally reverting smart contracts. Commonly used in gambling dApps.
Missing Permission Check	Improper validation of permissions when performing sensitive operations.
Pseudo Random Number Generators	Sensitive operations that trust the insecure implementation of pseudo-random number generators can cause security issues.
Suicidal	Unprotected interface allows the destruction of a victim's contract.
Call injection	An attacker is able to call a sensitive function.

^a A reentrant function is a function designed to be called recursively, or by two or more processes.

3. *Backward and forward references review*: for each included study, we analyzed both the citations and references to identify other relevant works;
4. *Works classification*: we performed a first examination to identify the primary contributions of each study and divided them into categories, as described in Table 3;
5. *Works review*: we reviewed the retrieved works in detail and developed the concepts presented in Sections 5 and 6.

Table 4 reports the overall number of publications that we reviewed at each step of the aforementioned process.

3.1. Automatic search

We search for eight relevant keyword pairs that relate security to WebAssembly: (i) “wasm security”; (ii) “wasm vulnerability”; (iii) “webassembly security”; (iv) “webassembly vulnerability”; (v) “wasm anomaly”; (vi) “webassembly anomaly”; (vii) “wasm detection”; (viii) “webassembly detection”.

We exclude other keywords, since a quick analysis shows that they basically provide almost equal results to the utilized keyword combinations. For instance, “cybersecurity” would provide the same results as “security”. The same holds true for the use of plural forms

Table 3
Security categories for the analyzed literature.

Category	Description	Nr. of studies
Security Analysis	Analyze the security of the WebAssembly ecosystem	5
Empirical Studies	Analyze a specific phenomenon from the security perspective in the WebAssembly ecosystem	6
Use case in cybersecurity	Leverage WebAssembly to realize a use case in the cybersecurity domain	30
Attack Scenarios	WebAssembly capabilities for realizing attack scenarios	9
Attack detection solutions	Detect attacks in WebAssembly programs	17
Vulnerability Discovery	Discover vulnerabilities in WebAssembly binaries	17
Security Enhancements	Extend WebAssembly language and components to increase security	32
Others	Relevant works to extend the literature of security works in WebAssembly	31

Table 4
Publications analyzed at each step of the literature review process.

Keywords	Nr. of papers from SCOPUS	Nr. of papers from IeeeXplore
“wasm security”	60	25
“wasm vulnerability”	27	11
“webassembly security”	118	55
“webassembly vulnerability”	43	15
“wasm anomaly”	5	2
“webassembly anomaly”	6	3
“wasm detection”	24	14
“webassembly detection”	61	24
	344	231
Merged and without duplicates		162
After inclusion and exclusion criteria		84
After reference analysis		147

such as “vulnerabilities” instead of “vulnerability”.

We use two relevant platforms containing peer-reviewed articles, i.e., IEEE Xplore [42] and Scopus [43]. After obtaining the works, we merge them and remove duplicates. Then, we apply the inclusion and exclusion criteria described further below, to select in-scope works only. Finally, we perform the backward and forward reference investigation to find other relevant works.

3.2. Works inclusion and exclusion criteria

We only include works specifically focused on the security domain. However, we include into a secondary category named “others” those works that we consider essential sources for investigating new security research topics in WebAssembly, even if they are not specifically focused on security. We primarily consider peer-reviewed articles, but during the backward and forward reference review, we also include relevant literature from pre-print services [44], as well as relevant technical papers from highly recognized international cyber-security conferences [45]. We exclude works not written in English and mention, in Section 5.8, works that explore security topics but do not provide satisfactory analyses, such as preliminary studies and poster papers.

3.3. Works classification

After the document retrieval phase, we first analyze each work’s objective to categorize and determine its primary contribution. We

delineate several categories and organize items based on their contribution type. Then, we go deeper into each paper to scrutinize its technical details or gain a better understanding of any proposed case studies by exploring the features of other works. Finally, we synthesize our analysis, as detailed in Section 5. For the specific category of malware detection, we exclude studies that discover vulnerabilities without involving WebAssembly features, e.g., crypto mining detection through machine-learning models on network traffic [46,47], or dynamic analysis focused on binaries not specifically generated by WebAssembly code [48]. In order to classify the reviewed works, we identify eight main categories:

- *Security Analysis*: papers that analyze the security of the WebAssembly ecosystem. These works are useful in providing preliminary context about security flaws that affect WebAssembly programs;
- *Empirical studies*: papers that investigate security aspects of WebAssembly by analyzing real-world scenarios. These works appport relevant empirical contributions;
- *Use cases in security*: in these papers, authors leverage WebAssembly features to enrich the security posture of a proposed solution, system, and/or framework;
- *Attack Scenarios*: these papers illustrate approaches that leverage WebAssembly capabilities for conducting complex attacks;
- *Attack detection solutions*: these works propose solutions to detect attacks on WebAssembly applications;
- *Security Enhancements*: in these papers, authors aim to extend WebAssembly components in order to mitigate security flaws that are examined in security analysis works;
- *Vulnerability Discovery*: the contribution of these papers is to provide approaches to discover vulnerabilities in applications based on WebAssembly;
- *Others*: some relevant papers are not specifically focused on cybersecurity but can be considered as preliminary studies representing “cornerstones” for conducting novel security-related research activities.

4. Related works

Several works offer pertinent reviews on WebAssembly that do not explicitly center on security.

WebAssembly runtimes are analyzed by Wang and Wenwen (2022) [49]. In their study, the authors review five predominant standalone WebAssembly runtimes and construct a benchmark suite for comparative purposes. They assert that performance is influenced by various factors and outline criteria for selecting the best runtime based on specific use cases. Additionally, they offer insights into enhancing runtime performance by including crucial information into compiled WebAssembly binaries.

In the realm of cloud-edge computing [50], Kakati et al. (2023) [51] provide a comprehensive review of the adoption of WebAssembly.

The Internet of Things (IoT) is another relevant context where WebAssembly is widely adopted [52]. Ray and Partha Pratim (2023) [53] provide a comprehensive survey about the adoption of WebAssembly in IoT systems by illustrating the potential of WebAssembly characteristics, providing a valuable resource for developers through a comparison of WebAssembly tools and toolchains, and illustrating future insights for continuing this research field.

4.1. Security studies

Section 5.1 d’ing security analysis for WebAssembly compilers, runtimes, and the impacts of porting programs written in unsafe memory languages [54–57]. Notably, Kim et al. (2022) [58] and Harnes and Morrison (2024) [59] provide a comprehensive survey on techniques and methods for WebAssembly binary security. They also propose

Table 5

Comparative analysis of WebAssembly surveys based on coverage of the eight categories identified above.

	Kim et al. (2022) [58]	Harnes and Morrison (2024) [59]	Our work
Security analysis			✓
Security enhancement	✓		✓
Vulnerability discovery	✓	✓	✓
Use case in cybersecurity			✓
Attack scenarios			✓
Attacks' detection	✓	✓	✓
Empirical studies	✓		✓
Other works			✓

Table 6

Number of analyzed publications for each category that is covered by both this work and related works.

	Kim et al. (2022) [58]	Harnes and Morrison (2024) [59]	Our work
Security enhancement	8	0	32
Vulnerability discovery	7	15	17
Attacks' detection	9	7	17

Table 7

Total number of reviewed works.

Kim et al. (2022) [58]	Harnes and Morrison (2024) [59]	Our work
24	22	147

future research directions to address open problems identified in their studies, such as protection from malicious WebAssembly binaries and hardening the inherent security of WebAssembly.

Tables 5, 6, and 7 present a comparative analysis of reviews. Our study extends beyond the scope of previous works by encompassing various types of literature, including those focused on enhancing the security of WebAssembly components, utilizing WebAssembly for security purposes in various use cases, conducting empirical studies, and performing security analyses. When compared to existing surveys, we actually cover five additional categories of interest, culminating in the analysis of 147 works.

It is worth noting that we classified certain specific publications [60, 61] under the generic category “other”, rather than grouping them with related studies. Such a decision was based on our analysis, which found that these publications lacked sufficient information to be classified as vulnerability discovery efforts.

To the best of our knowledge, this is the first work providing a comprehensive review of security research in the WebAssembly domain.

5. WebAssembly and security: a review

This section presents the works analyzed in our study. As previously outlined, Table 3 summarizes the categories detailed in Section 3, and displays the number of analyzed studies for each of them.

5.1. Security analysis

Although WebAssembly has been designed with security in mind, relevant works underline that it can suffer from security issues. One significant study conducted by Lehmann et al. (2020) [54] highlights this concern. In their research, the authors demonstrate that certain classic vulnerabilities, which are mitigated by compiler system enhancements in native binaries, remain exploitable in WebAssembly. They conduct an extensive security analysis of WebAssembly’s linear memory system, present examples of vulnerable applications, outline attack primitives, demonstrate end-to-end exploits for identified vulnerabilities, and propose possible mitigation strategies to harden WebAssembly binaries. In

particular, the authors analyze the impact of classic attack primitives and categorize them into three main types: write operations (including stack-based buffer overflows [62], stack overflows, and heap metadata corruptions), data overwrites (such as overwriting stack and heap variables, as well as “constant” data), and triggers for unexpected behavior (such as redirecting indirect calls, injecting code into the host environment, and overwriting application-specific data). They illustrate how these attack primitives can be combined to execute end-to-end attacks such as cross-site scripting, remote code execution, and arbitrary file writing.

To mitigate the analyzed security flaws, the authors propose three possible approaches: extending the WebAssembly language, enhancing compilers and toolchains, and developing robust and safe libraries for use during development.

Stiévenart et al. (2021) [55] explore the security vulnerabilities stemming from the absence of protections in WebAssembly compilers. In their study, the authors compile 4,469 C programs containing known buffer overflow vulnerabilities (both stack-based and heap-based) to x86 code and WebAssembly. They reveal that 1,088 programs exhibit a different behavior when compiled to WebAssembly. In particular, WebAssembly is found to be more vulnerable to stack-smashing attacks due to the absence of security measures like stack canaries.

As illustrated before, memory management in low-level programs is the primary cause of security flaws. Therefore, the most affected programming language is C [63], as it provides the lowest abstraction level with respect to the underlying machine so that it can fully interact with the hardware.

Stiévenart et al. (2022) [56] investigate the security implications of cross-compiling C programs to WebAssembly. In their study, the authors compile and execute 17,802 programs with common vulnerabilities for both 64-bit x86 and WebAssembly. They find that 4,911 binaries yield different results due to three main reasons: the absence of security measures in WebAssembly, different semantics of execution environments, and variations in used standard libraries. Critical differences affecting security include the implementation of `malloc()` and `free()` functions, and the lack of both stack-smashing and memory protection mechanisms. To address these issues, the authors suggest extending WebAssembly to incorporate memory semantics in C and C++ code. They also emphasize the importance of static analysis in identifying security flaws and highlight related works aiming to enhance the WebAssembly runtime and mitigate the security impacts [64–66].

Porting vulnerable C code to WebAssembly may introduce security bugs, as illustrated by Massidda et al. (2024) [67]. In their study, authors analyze six WebAssembly vulnerabilities by revealing that memory-related vulnerabilities in C programs potentially lead to critical issues like Cross-Site Scripting (XSS) and Remote Code Execution (RCE) when ported to WebAssembly.

One of the noteworthy resources for developers is the technical study conducted by Brian McFadden et al. (2018) [57]. In this work, the authors delve into various facets of WebAssembly security. In particular, they:

- analyze Emscripten’s implementation of compiler and linker-level exploit mitigation;
- introduce new attack vectors and methods of exploitation in WebAssembly, demonstrating that buffer overflow or indirect function calls can lead to cross-site scripting or remote code execution attacks;
- provide examples of memory corruption exploits in the WebAssembly environment;
- outline best practices and security considerations for developers aiming to integrate WebAssembly into their products.

5.2. Empirical studies

These works present empirical case studies to analyze a specific phenomenon in the WebAssembly ecosystem. Early studies analyze the wide-spread use of WebAssembly for executing cryptojacking attacks in the wild.

Rüth et al. (2018) [68] introduce a new fingerprinting method aimed at identifying miners, which revealed a factor of 5.7% more miners compared to publicly available block lists. They also classify mining websites among 138 million domains sourced from the largest top-level domains and the Alexa top 1 million websites [69]. The authors demonstrate that the prevalence of browser mining is low, at less than 0.08%, and that 75% of the mining sites utilize Coinhive, a web-based mining provider.

Musch et al. (2019) [70,71] analyze websites from Alexa top one million ranking [69] in detail and find that 1 out of 600 websites uses WebAssembly. However, they reveal that 50% of all sites using WebAssembly employ it for mining and obfuscation purposes.

Hilbig et al. (2021) [18] also analyze the usage of WebAssembly worldwide, presenting different results. They collect 8,461 WebAssembly binaries from a wide range of sources and conclude that:

- 64.2% of the binaries are compiled from memory-unsafe languages, such as C and C++;
- less than 1% of all binaries are related to crypto mining, with a greater diffusion of game-based, text processing, visualization, and media applications;
- 29% of all binaries are minified, suggesting a need for approaches to decompile and reverse engineer WebAssembly.

Romano et al. (2021) [72] conduct two empirical studies to perform a qualitative analysis of 146 bugs related to WebAssembly in Emscripten and a quantitative analysis of 1,054 bugs in three open-source compilers, namely, AssemblyScript, Emscripten, and Rustc/Wasm-Bindgen. They draw the conclusion that sensitive information can be disclosed in Trusted Execution Environments (TEE). Puddu et al. (2024) [73] analyze the impacts of executing confidential code on top of a WebAssembly runtime within a TEE and show that using an intermediate representation such as WebAssembly can lead to code leakage.

Waseem et al. (2024) [74] provide a relevant empirical study that analyzes 385 issues and root causes on 12 open-source WebAssembly applications deployed on GitHub and 354 “question-answer” posts on Stack Overflow. The work is not merely focused on security and aims to classify 739 issues into nine different types. The authors identify 29 security issues (4.83% of all issues) related to authentication, cryptographic, and compliance concerns caused by the specific WebAssembly structure and execution model. The authors underline that while WebAssembly is a secure alternative to JavaScript, it is crucial to develop robust security practices to improve the overall resilience to attacks.

5.3. Use cases in cybersecurity

WebAssembly is extensively used in heterogeneous domains where security is a crucial non-functional requirement. The following subsections summarize the most interesting contributions we found in the literature by underlying the main research paths they have tried to follow.

5.3.1. Browser extensions

WebAssembly was designed to provide an assembly language for the web, especially for modern browsers. Therefore, there is extensive literature about realizing browser extensions with this technology. As WebAssembly is a portable language, all the works presented in this section could potentially be ported to browser-based environments. However, the works that we classify as “Browser extensions” have specifically been realized to extend browser capabilities and increase portability, isolation, and efficiency.

Baumgärtner et al. (2019) [75] present a Disruption-tolerant Networking (DTN) system for browsers that leverages WebAssembly and the Bundle Protocol 7 draft implementation [76]. Narayan et al. (2020) [77] define a new framework (RLBox) that implements fine-grained isolation for sandboxing third-party libraries. The framework has been integrated into production Firefox to sandbox the libGraphite font shaping library. Chen et al. (2023) [78] propose a novel approach for digital rights management of streaming. The work allows users to easily fetch videos with a plug-and-play approach and enhances decryption efficiency by minimizing interactions with the server.

WebAssembly in the browser is employed by Jang et al. (2022) [79] to realize a distributed public collaborative fuzzing system for the security of applications emulated inside the browser engine.

5.3.2. Cryptography, trusted execution environments and privacy

Various works aim to safeguard data integrity and confidentiality through cryptographic techniques, Trusted Execution Environments, and privacy-preserving methods. These approaches are tailored to specific domains and can be implemented in hardware, embedded systems, or browsers. Sun et al. (2020) [80] leverage WebAssembly and the Web Cryptography API to devise a client-side encrypted storage system capable of storing and sharing data across heterogeneous platforms. Attrapadung et al. (2018) [81] use WebAssembly to implement an efficient two-level homomorphic public-key encryption within prime-order bilinear groups. Seo et al. (2023) [82] introduce a portable and efficient WebAssembly implementation of the Crystals-Kyber post-quantum Key Encapsulation Mechanism [83]. Zhao et al. (2023) [84] define a novel approach for constructing reusable, rapid, and secure enclaves. They introduce three techniques, namely, enclave snapshot and rewinding, nested attestation, and multi-layer intra-enclave compartmentalization, for enabling rapid enclave reset and robust security. WebAssembly is used to realize publish and subscribe environments for cloud-edge computing. Both Almstedt et al. (2023) [85] and Ménétrey et al. (2024) [86] design a novel publish and subscribe middleware running in Intel SGX for trustworthy and distributed communications. Qiang et al. (2018) [87] utilize the sandbox properties of WebAssembly combined with Intel SGX enclaves to develop a two-way sandbox. This approach offers dual protection: safeguarding sensitive data from potential threats posed by the cloud provider and shielding the host runtime from potential attacks initiated by malicious users. Other works show that WebAssembly can be used to implement a high-performance, memory-safe, and portable client-side-hashing library (Riera et al. (2023) [88]), or even a fully-fledged client-side storage platform (Sun et al. (2022) [80]).

Coppolino et al. (2024) [89] propose a trustworthy multi-tenant model for embedded systems which integrates WebAssembly with a Trusted Execution Environment to ensure isolation. This model also includes a patched WebAssembly System Interface for safe system call execution and a monitoring layer for anomaly detection.

Modern developments in the blockchain domain aim to leverage WebAssembly features for executing smart contracts. To address the upcoming upgrade in Ethereum 2.0, which involves replacing the Ethereum Virtual Machine with WebAssembly for smart contract interpretation, Chen et al. (2024) [90] propose a novel EVM2WASM bytecode translation framework capable of translating real-world smart contracts with high fidelity and minimal gas overhead.

5.3.3. Identity and access management

Another integration of Intel SGX and WebAssembly is provided by Goltzsche et al. (2019) [91]. In their work, the authors present AcCTEE, a framework for realizing fine-grained permissions for resource accounting by preserving the confidentiality and integrity of code and data. Also, Zhang et al. (2021) [92] propose EL PASSO, a portable and “zero-configuration” module to allow the setup and configuration of privacy-preserving single-sign-on operations.

Table 8

Works focused on describing attack examples in WebAssembly.

Cite	Type of implemented attack
Genkin et al. (2018) [102]	Side-channel attacks
Easdon et al. (2022) [103]	Side-channel attacks
Rokicki et al. (2022) [104]	Side-channel attacks
Katzman et al. (2023) [105]	Side-channel attacks (cache)
Cabrera-Arteaga et al. (2023) [106]	Malware detection evasion
Romano et al. (2022) [107]	Malware detection evasion
Cao et al. (2023) [108]	Malware detection evasion
Loose et al. (2023) [109]	Malware detection evasion
Oz et al. (2023) [110]	Ransomware attack

5.3.4. Cloud, cloud-edge computing and IoT

Cloud and cloud-edge computing are common use cases of WebAssembly adoption [93]. Edgedancer [66] is a platform for supporting portable, provider-independent, and secure migration of edge services. Ménétrey et al. (2022) [94] envisage that WebAssembly with TEEs is a promising combination to realize cloud-edge continuum [95].

Sun et al. (2022) [80] primarily focus on the browser domain as a platform aimed at realizing a secure cloud storage environment. This subject is also addressed by Song et al. (2023) [96], who provide a cross-platform secure storage architecture through the realization of a new primitive called Controllable Outsourced Attribute-Based Proxy Re-Encryption (COAB-PRE) alongside the cross-platform capabilities of WebAssembly. Although WebAssembly provides several security benefits, performance overheads should be investigated when it is adopted in contexts with constrained resources, such as IoT and edge devices. Wen et al. (2020) [97] address this problem by defining a novel operating system that integrates Rust and WebAssembly to make IoT applications more efficient and secure. A similar approach is also proposed by Li and Sato (2023) [98], who implement a kernel focused on isolation, security, and customizability through the combination of WebAssembly and Rust.

An interesting recent work by Karanjai and Shi (2024) [99] leverages WebAssembly for executing trusted Large Language Model inference through the blockchain technology and across edge devices.

5.3.5. Face recognition systems

Preliminary work has been conducted on using WebAssembly to realize face recognition systems. Indeed, WebAssembly ensures a client-side solution that preserves image privacy and could potentially reduce the costs of adopting external services. Pillay et al. (2019) [100] provide a real-time browser-based application that is able to attain a facial recognition rate of 91.67%, while Martín Manso et al. (2021) [101] compare JavaScript, WebGL and WebAssembly for face detection, and conclude that WebAssembly strikes a good balance in terms of detection accuracy and computation time overhead.

5.4. Attack scenarios

Another research field deals with innovative approaches for conducting attacks against WebAssembly programs. Most articles fall under two primary categories:

- *Side-channel attacks*: these works demonstrate how it is possible to conduct side-channel attacks to disclose sensitive keys or realize covert channels [102–105];
- *Evasion techniques for malware detection*: these works illustrate in which way it is possible to bypass malware detection systems through WebAssembly [106–109].

Oz et al. (2023) [110] provide a divergent work by showing how it is possible to leverage WebAssembly to implement ransomware attacks [111].

Table 8 summarizes the above information.

5.4.1. Side-channel attacks

Genkin et al. (2018) [102] utilize WebAssembly to induce CPU cache contention and extract sensitive data from other programs' memory accesses. They demonstrate that this technique can extract sensitive keys from famous cryptographic libraries. An innovative cache-based attack is also illustrated in the work of Katzman et al. (2023) [105], where authors conclude that transient execution can increase the effectiveness of cache attacks.

Easdon et al. (2022) [103] leverage WebAssembly features to demonstrate a novel technique for prototyping microarchitectural attacks. They achieve this by implementing two open-source frameworks, *libtea* and *SCFirefox*, and developing proof-of-concepts for executing Foreshadow [112] and Load Value Injection (LVI) [113] attacks.

Rokicki et al. (2022) [104] introduce the first port contention side-channel attack conducted entirely within the browser environment. Their study demonstrates that certain WebAssembly instructions can induce port contention attacks, enabling the extraction of sensitive data from concurrent processes executing on the same CPU core.

5.4.2. Malware detection evasion

Two other works propose innovative techniques to evade malware detection in WebAssembly binaries. Cabrera-Arteaga et al. (2023) [106] analyze the potential of WebAssembly binary diversification to circumvent detection by antivirus systems. On the other hand, Loose et al. (2023) [109] introduce a novel approach to embed adversarial payloads into the instruction stream of binaries, thereby bypassing malware detection systems.

Additionally, Romano et al. (2022) [107] demonstrate the feasibility of obfuscating JavaScript to evade malware detection systems by converting specific JavaScript instructions into WebAssembly. Cao et al. (2023) [108] develop a framework for static binary rewriting, which can be used for various purposes such as obfuscation. Although the primary focus is not solely on evading malware detection systems, the framework can be adapted for that purpose, as well as for patching vulnerabilities or binary instrumentation.

5.5. Attack detection solutions

These works focus on detecting attacks in WebAssembly programs, primarily targeting cryptojacking attacks. As shown in Table 9, current research is essentially focused on detecting crypto-mining activities facilitated by the WebAssembly technology.

5.6. Security enhancements

Several studies have extended WebAssembly components or integrated new systems to enhance their security level. Some authors have extended the semantics of the WebAssembly language to include protection against various threats. For instance, Watt et al. (2019) [132] propose CT-Wasm, a type-driven secure language for writing cryptographic libraries capable to protect against time-based side-channel attacks. Similarly, Michael et al. (2023) [133] focus on securing WebAssembly program memory by introducing MSWasm, a framework that addresses memory safety vulnerabilities through semantic analysis. They introduce a new memory region called *segment memory* and new instructions for interacting with it. Additionally, Geller et al. (2024) [134] extend WebAssembly semantics to define indexed types [135] capable to ensure safety properties over program values. Another language extension aimed at ensuring memory safety is proposed by Zhang et al. (2023) [136], who allow the definition of canaries in the program.

Several works modify the WebAssembly compiler to formally verify security properties [137,138] or reduce the attack surface in embedded systems [139,140]. Others focus on enhancing the WebAssembly runtime environment to create a trusted execution environment that increases isolation and prevents memory attacks during execution.

Table 9

Works about detecting attacks against WebAssembly applications.

Work	Detection approach	Type of detected attacks
Konoth et al. (2018) [114]	Runtime detection on the basis of three approaches discovered through static analysis of WebAssembly malicious binaries.	Cryptojacking based on hijacking user's CPU for mining purposes.
Wang et al. (2018) [115]	At runtime, monitor execution through in-line scripts.	XSS-assisted cryptojacking and mining.
Rodriguez and Possega (2018) [116]	Analyze WebAssembly and JavaScript APIs, and train a Support Vector Machine (SVM). Show the feasibility of deploying the solution as a browser extension.	Cryptojacking based on API calls and Web Worker abuse.
Kharraz et al. (2019) [117]	Support Vector Machine classifier.	Cryptojacking based on cryptographic operations and web worker creation for mining purposes.
Bian et al. (2020) [118]	At runtime.	Cryptojacking for mining purposes, using CPU throttling and resource obfuscation.
Kelton et al. (2020) [119]	At runtime, through a Support Vector Machine (SVM) classifier.	Cryptojacking for mining purposes, abusing victim's computing resources.
Mazaheri et al. (2020) [120]	Static analysis by analyzing features of web pages that indicate the presence of time-based side-channel attacks.	Side-channel.
Romano et al. (2020) [121]	Static analysis, by reconstructing binaries in "standard" WebAssembly format, and analyzing the generated intra-procedural and inter-procedural Control-Flow Graphs (CFGs).	Cryptojacking involving stealthy techniques for mining purposes.
Yu et al. (2020) [122]	Static analysis with machine learning and runtime detection systems. Provide both detection and blocking capabilities.	Cryptojacking at the proxy server level for distributed mining.
Naseem et al. (2021) [123]	At runtime, leveraging a Convolutional Neural Network (CNN) model trained with a comprehensive dataset of both malicious and benign WebAssembly binaries.	Malicious cryptojacking WebAssembly binaries.
Tommasi et al. (2022) [124]	At runtime, browser extension based on a Support Vector Machine (SVM) classifier.	Cryptojacking, focused on detecting hidden web-based crypto-mining in the browser.
Khan Abbasi et al. (2023) [125]	Combine blacklist-based techniques, static analysis, and feature analysis to identify malicious website scripts.	In-browser unknown cryptojacking malware variants based on WebAssembly and/or non-WebAssembly code.
Breitfelder et al. (2023) [126]	Static WebAssembly analysis that is able to capture call-, control-, and data-flow graphs for further analysis. The detection capability is one of the discussed use-cases.	Obfuscated cryptomining payloads.
Komiya et al. (2024) [127,128]	Provide a tool to generate a dataset of malicious WebAssembly binaries and leverage Natural Language Processing vectorization models based on Doc2Vec and SVM classifiers for detection.	Attack vectors involving malicious websites.
Helpa et al. (2024) [129]	Leverage the Debugging With Attributed Record Format (DWARF) to obtain relevant information for training and testing four machine learning algorithms for classification.	Potentially Unwanted Applications (PUA) [130] in WebAssembly binaries.
Heinrich et al. (2024) [131]	Categorize data extracted from WASI calls during its execution to identify anomalies through machine learning models.	The work refers to 18 types of attack samples. ^a

^a It is unfortunately highly generic with respect to describing the specific types of attacks under examination.

In order to protect the execution of WebAssembly programs, several authors have designed additional mechanisms. For example, Sun et al. (2019) [141] propose SELWasm, a code protection technique that implements a self-checking mechanism to prevent code execution on unauthorized websites, along with an "Encryption & Decryption" approach for ensuring code confidentiality. Similarly, Brandl et al. (2023) [142] present a novel interpreter for WebAssembly capable of

performing complex analyses such as inter-procedural control and data-flow information. This study can be considered as a foundation for future static security analysis approaches.

Additionally, Pop et al. (2022) [143] propose an interpreter that extends WASMI 14 [144] by adding pausing, serialization, deserialization, and resumption primitives for implementing a portable enclave service.

Gao et al. (2024) [145] leverage the “zero-knowledge Succinct Non-interactive Argument of Knowledge (ZKSNARK)” [146] technology to realize a hardened virtual machine that emulates the execution of WebAssembly bytecode to generate zero-knowledge-proofs for the emulation result.

Other works explore novel approaches to obfuscate WebAssembly binaries, aiming to increase protection against reverse engineering or to highlight the adversarial capabilities of WebAssembly in evading malware detection systems.

Additional works introduce verification of programs after compilation. Watt (2018) [147] mechanizes and verifies the WebAssembly specification using the theorem prover Isabelle [148], illustrating two demonstrations that implement a verified type-checker and interpreter for WebAssembly. This work is extended by Watt et al. (2023) [149], who define a WebAssembly interpreter written in Isabelle/HOL [150] to validate the Wasmtime interpreter [151]. Johnson et al. (2021) [152] introduce VeriWasm, a static offline verifier for native x86-64 binaries compiled to WebAssembly. The verifier checks the boundaries of memory accesses, control flows, and stack usage, ensuring the satisfaction of Software Fault Isolation (SFI) [153] in binaries. Watt et al. (2021) [154] extend the WebAssembly semantic and provide two different theorem provers: WasmCert-Isabelle and WasmCert-Coq. VanHattum et al. (2024) [155] present Crocus, a system for lightweight, modular verification of instruction-lowering rules within “Craneflight” [156], a compiler backend used by the Wasmtime runtime. Legoupil et al. (2024) [157] extend the verification process to MSWasm, the WebAssembly extension provided by Michael et al. (2023) [133].

Another area of research focuses on improving malware detection in WebAssembly binaries. Xia et al. (2024) [158] highlight the issue of emerging JavaScript-WebAssembly multilingual malware (JWMM), which reduces the effectiveness of antivirus solutions. The authors introduce a novel approach to semantically reconstruct WebAssembly binaries, generating a high-level structure aimed at enhancing antivirus detection capabilities. Huang and Zhao (2024) [159] present a multi-modal pre-trained language model for WebAssembly reverse engineering. The framework can be fine-tuned with a few labeled data to achieve specific reverse engineering tasks.

5.6.1. Works that enhance WebAssembly indirectly

Some works do not extend WebAssembly components directly but address security issues by enhancing other systems. Schrammel et al. (2020) [160] improve WebAssembly security indirectly by enhancing the JavaScript (JS) V8 engine. The authors underline that the JS engine compiles some JavaScript parts in WebAssembly and identify several potential attacks against the compiled code. They propose a JS engine extension that provides isolation mechanisms using modern CPUs to enforce domain-based access controls on memory pages. Vassena et al. (2021) [161] introduce Blade, an approach to automatically eliminate speculative leaks from cryptographic code by extending “Craneflight” [156]. Song et al. (2023) [162] propose an interesting method for blocking heap memory corruption attacks by shadowing metadata from linear memory in trusted JavaScript memory. They provide a JavaScript API that WebAssembly programs can call to shadow and validate the metadata of the used linear memory. In their study, Narayan et al. (2024) [163] describe the Hardware-assisted Fault Isolation (HFI) extension to processors for addressing the limitations of existing software-based fault isolation (SFI) and underline the seamless integration with existing SFI techniques such as WebAssembly.

5.7. Vulnerability discovery

Several works aim to discover vulnerabilities in WebAssembly binaries. Tables 10 and 11 categorize these efforts into two groups: (i) works that leverage static security analysis and (ii) works proposing dynamic analysis. Each approach is experimented with various

techniques, allowing the discovery of different types of vulnerabilities.

Many studies focus on identifying issues in smart contracts. This clearly indicates a significant adoption of WebAssembly in the blockchain domain.

A comparative analysis of the mentioned works allows us to observe that there is no standardized approach to comparing the effectiveness of the proposed methods. While some works focus on identifying low-level vulnerabilities such as integer and stack-based overflows, others target vulnerabilities specific to the smart contract domain. Some authors use real-world applications for their evaluations, while others employ well-known yet heterogeneous datasets. Evaluating real-world applications effectively demonstrates the capability of security scanners to discover new vulnerabilities. However, this approach does not yield performance metrics such as false positives or negatives, as the exact number of vulnerabilities in the applications under test is unknown. Consequently, each author defines different performance metrics, making it challenging to compare the proposed methods.

This variability in evaluation metrics is a common issue in vulnerability discovery approaches [181], and we encourage future research to address this open problem.

5.8. Other works

Many works focus on aspects of WebAssembly beyond security. Though, they are still relevant as they can provide valuable insights for future efforts aimed at enhancing the security of the WebAssembly domain.

5.8.1. Preliminary works

Some preliminary studies offer promising results that may serve as a foundation for innovative security improvements. Abbadini et al. (2023) [182] design an approach that combines eBPF features with WebAssembly to enhance host isolation security in existing runtimes. Narayan et al. (2021) [183] provide a tutorial on leveraging WebAssembly’s sandboxing feature to run unsafe C code securely. Stiévenart and De Roover (2021) [60] introduce “Wassail”, one of the first static analysis tools for WebAssembly. Their technology enables several static analyses, such as call graphs, control-flow graphs, and dataflow analysis. Cabrera et al. (2021) [61] present the first version of a code diversification framework for WebAssembly, which is further extended in a more recent study [184].

Bhansali et al. (2022) [185] provide a preliminary overview of applying obfuscation techniques to WebAssembly binaries, demonstrating that specific obfuscation approaches can bypass the MINOS cryptojacking detector (Naseem et al. (2021) [123]).

Park and Cho (2024) [186] leverage the Bitwise Memory Operation (BMO) technique for memory protection and vulnerability mitigation purposes.

Zheng et al. (2023) [187] present a software prototype for dynamic program analysis capable of detecting memory bugs and integer overflows in WebAssembly code, serving as an essential reference for investigating vulnerability discovery in this domain.

Tinto and Tullio (2024) [188] leverage the portability and isolation capabilities of WebAssembly, along with the memory safety features of the Rust programming language, to propose a preliminary design for a runtime infrastructure supporting the Continuum of Computing.

5.8.2. Works about vulnerability discovery approaches

Many works investigate novel approaches to inspect WebAssembly code and binary structure, laying the groundwork for future vulnerability discovery research. William Fu et al. (2018) [189] and Szanto et al. (2018) [190] provide comprehensive overviews of taint tracking [191] in WebAssembly. Both studies demonstrate promising results in tracking data flows within WebAssembly binaries but do not provide detailed security evaluations. Therefore, further research is needed to assess their effectiveness in real-world scenarios.

Table 10
Vulnerability discovery works that propose a static analysis approach.

Work	Static analysis approach	Discovered vulnerabilities
Quan et al. (2019) [164]	Control Graph Flow analysis	Fake EOS transfer, fake EOS notice
Yang et al. (2020) [165]	Symbolic Semantic Graph	Integer overflow, pseudo-random number generator, insecure message call
Li et al. (2022) [166]	Context-sensitive data flow analysis	Fake EOS Transfer, Forged Transfer Notification, and Block Information Dependency
Brito et al. (2022) [167]	Code property graph	Buffer overflow, use after free, double free, dangerous function usage, and format strings vulnerabilities
Scherer et al. (2024) [168]	Sound reachability analysis by encoding the WebAssembly semantic in Horn clauses	Arbitrary memory write vulnerabilities, integer overflows, sensitive data overwrites, general programmer-defined assertion failures, host imported function abuse
Tu et al. (2023) [169]	State Machine Representation	Rollback, missing permission check, integer overflow

Table 11
Vulnerability discovery works that propose a dynamic analysis approach.

Work	Dynamic analysis approach	Detected vulnerabilities
Huang et al. (2021) [170]	Black-box fuzzer	Block information dependency, forged transfer notification, fake EOS transfer
Jiang et al. (2021) [171]	Symbolic execution	Greedy dangerous DelegateCall, block information dependency, reentrancy, mishandled exception
He et al. (2021) [172]	Symbolic execution	Fake EOS, fake Receipt, rollback, missing permission check
Daniel Lehmann et al. (2021) [173]	Fuzzing	Crashes in binary programs, mainly related to buffer overflow conditions
Marques et al. (2022) [174]	Concolic execution	Not specified
Yang et al. (2022) [175]	Tracing and control-data flow	Access control vulnerabilities
Khan et al. (2023) [176]	Fuzzing	Data races, null pointer dereferences, out-of-bound accesses, division-by-zero errors in SGX enclave applications
Jin et al. (2023) [177]	Fuzzing	Overflow, underflow, arbitrary value transfer (e.g., reentrancy), suicidal call injection
Haßler et al. (2022) [178]	Coverage-guided fuzzing	Crashes in binary programs
Chen et al. (2022) [179]	Concolic execution	Fake EOS transfer, fake notification, missing authorization verification, blockinfo dependency, and rollback vulnerabilities
Zhou et al. (2023) [180]	Bytecode instrumentation, run-time validation, and grey-box fuzzing	Integer and stack overflow

Stiévenart et al. (2022) [192] and Stiévenart et al. (2023) [193] introduce methods to minimize WebAssembly binaries through slicing. These approaches are useful for optimizing dynamic and dependency analysis of WebAssembly binaries.

Cabrera-Arteaga et al. (2024) [184] demonstrate how to automatically transform a WebAssembly binary into variants without altering its original functionality. This approach is effective for fuzzing WebAssembly compilers. In such a field, a relevant research advancement

is provided by Zhang et al. (2024) [194], who propose a novel fuzzing approach for discovering security issues affecting WebAssembly compilers, while Han et al. (2024) [195] propose an efficient method for fuzzing WebAssembly interpreters.

Other works can contribute to the realization of static analysis approaches. Notable examples are Wasm Logic, a program logic for first-order WebAssembly proposed by Watt et al. (2019) [196], and “Eunomia” (He et al. (2023) [197]), a symbolic execution engine for WebAssembly.

Stiévenart et al. (2020) [198] present a novel procedure based on static analysis of WebAssembly programs based on compositional information flow. This method enables an efficient and precise summarization of function behaviors, facilitating the detection of information flow violations and potential security breaches. He et al. (2024) [199] propose an efficient and fully functional symbolic execution engine that has been adopted to identify more than 30 0-day vulnerabilities in well-known C, Go, and SGX applications.

Lehmann et al. (2019) [200] contribute to the dynamic analysis of WebAssembly with Wasabi, a framework designed for dynamically analyzing WebAssembly programs to collect runtime information, thus enabling the detection of dynamic behaviors such as memory accesses and control flow.

An interesting WebAssembly use case is proposed by Kotenko et al. (2023) [201], who devise practical guidelines for implementing secure applications in the power energy sector. Even if WebAssembly is merely mentioned, this work can pave the ground for future research.

Namjoshi et al. (2021) [202] define an interesting compiler extension that checks for program correctness against an independent proof validator. Although the proposed contribution is related to revealing bugs that occurred in the optimization process of compiled programs, it could be easily extended in order to also include security verifications.

To identify bugs in WebAssembly runtimes, Zhou et al. (2023) [203] propose WADIFF, a differential testing framework that could be specialized to find security bugs. Zhang et al. (2023) [204] propose another relevant work that characterizes the WebAssembly runtime bugs by collecting 311 real-world bugs from hundreds of related posts on GitHub, identifying 31 bug categories and related fixing strategies. In their work, authors develop a pattern-based bug detection framework to automatically detect bugs in WebAssembly runtimes.

Code debugging still remains an effective approach to manually discover vulnerabilities [205]. Lauwaerts et al. (2022) [206] extend the WARDuino WebAssembly microcontroller virtual machine [207] to enable debugging features. Future works may extend this by proposing security methodologies to find vulnerabilities in embedded systems designed with WebAssembly through debugging approaches.

5.8.3. Comparison with other works

Another category of works compares WebAssembly security with other technologies. Dejaeghere et al. (2023) [208] compare the security features of WebAssembly with those provided by eBPF, a Linux subsystem that allows the safe execution of untrusted user-defined extensions inside the kernel [209]. They demonstrate that different threat models can be defined for these two technologies and emphasize that WebAssembly's design focuses more on security than performance. While WebAssembly permits the execution of unsafe code, unlike eBPF, it reduces security risks and exploitation opportunities by limiting the number of implemented helper system libraries.

In their study, Pham et al. (2023) [210] assess several non-functional requirements, including performance, maintainability, and security. They demonstrate that WebAssembly is a viable alternative to Docker containers in IoT applications. However, the security evaluation is not exhaustive, suggesting the need for further studies to analyze the security disparities between these technologies.

Bastys et al. (2022) [211] present a significant contribution with their SecWasm framework, designed for general-purpose information-flow control in WebAssembly. SecWasm can be instrumented to perform several relevant security controls against WebAssembly programs. While the paper provides a comprehensive description of the approach, it lacks evaluations. Future research could explore and analyze this promising approach further.

In the realm of kernel isolation, Peng et al. (2023) [212] introduce a novel kernel context isolation system that offers enhanced isolation functionality and features an efficient "implicit" context switch to minimize performance overhead. Although the paper compares the system to WebAssembly to demonstrate isolation benefits, further evaluation could enhance understanding. Sun et al. (2024) [213] present a declarative approach to secure IoT applications using TrustZone, and compare it to the state-of-the-art WaTZ [214] framework.

6. Discussion

In this section, we discuss the results of our survey and summarize the characteristics of the analyzed works.

Our research reviewed 147 works. Of these, 116 (78.91%) have been classified into seven categories. The remaining 31 (21.09%) works do not make significant contributions to the security domain in WebAssembly, yet represent relevant references for novel pathways in related research topics.

Fig. 1 shows the percentage of works for each category. As it is possible to observe, most contributions (29.1%) aim to harden WebAssembly security. Another large set of works (21.8%) leverages the WebAssembly capabilities for implementing security-based use cases. The other relevant areas are related to vulnerability discovery (15.5%) and attack detection (15.5%), respectively.

6.1. WebAssembly security

Studies that explore the security of WebAssembly through analyses, empirical studies, or by showing attack scenarios led to the following primary conclusions:

- Although WebAssembly was designed to provide better isolation and security, the lack of security protection mechanisms in its semantics and compilers has the opposite effect, increasing the risks of memory-based attacks by porting unsafe programs developed with low-level languages such as C into WebAssembly;
- This problem is amplified by the presence of a large number of WebAssembly binaries derived from memory-unsafe languages in real-world applications;
- WebAssembly should provide isolation and protect sensitive data from disclosure, but several studies confirm that it is possible to extract sensitive data through specific techniques, such as side-channel attacks based on cache analysis or port contention;

Table 12

Security categories and publicly available works.

Category	Relevant works	Public repositories
Security Analysis	5	2
Empirical studies	6	6
Use case in cybersecurity	30	6
Attack Scenarios	9	3
Attacks' detection	17	4
Security Enhancements	32	20
Vulnerability Discovery	17	9

- A significant number of WebAssembly binaries in the wild are obfuscated, underscoring the need to expand research into reverse engineering and WebAssembly decompilation;
- The obfuscation applied to WebAssembly has led to a performance reduction in malware detection systems, highlighting the importance of exploring the research field of WebAssembly binary reverse engineering;
- The relevance of crypto-mining activities based on WebAssembly in the world is a contested matter: while some studies underline that it is the primary usage of such technology, others state that less than 1% of the analyzed binaries are related to crypto-mining activities.

6.2. Detect attacks and discover vulnerabilities in WebAssembly

Detection efforts primarily focus on cryptojacking attacks, while vulnerability discovery approaches mainly address vulnerabilities in smart contracts. Consequently, there is significant interest in analyzing WebAssembly applications in smart contract environments. As stated in Section 2.2.3, there are relevant use-cases in web applications, data visualization, and gaming. Future works should analyze the security impact of WebAssembly in these other domains and propose approaches for addressing security problems.

Another significant issue is the heterogeneity of used benchmarks and performance metrics. Some authors merely analyze their accuracy, and no common dataset is used for trials and experiments.

The Alexa top one million website benchmark [69] was extensively employed as "ground truth" for representing real-world WebAssembly cases. Unfortunately, such a source was retired in 2022 and is unusable for further studies. We encourage researchers to expand their scope by including additional sources to create a more reliable ground truth benchmark.

6.3. The interest of WebAssembly for cybersecurity use-cases

Fig. 2 shows the number of works published for each security-related domain and the reasons behind the WebAssembly adoption. As observed, WebAssembly is utilized across various application types, with particular relevance in cloud and cryptography domains. WebAssembly is chosen for several reasons, notably performance, isolation, and portability. Some studies also highlight its adoption to ensure memory safety. However, as previously stated, several studies confirm security gaps in using WebAssembly for both isolation and memory safety, underscoring the necessity for further investigation into the security of these applications.

6.4. Enhance WebAssembly security

Fig. 3 reports a heatmap illustrating the number of works extending specific components of WebAssembly to enhance their security properties.

What can be inferred from the heat map is that most of the works focus on memory operations and involve modifications to the compiler

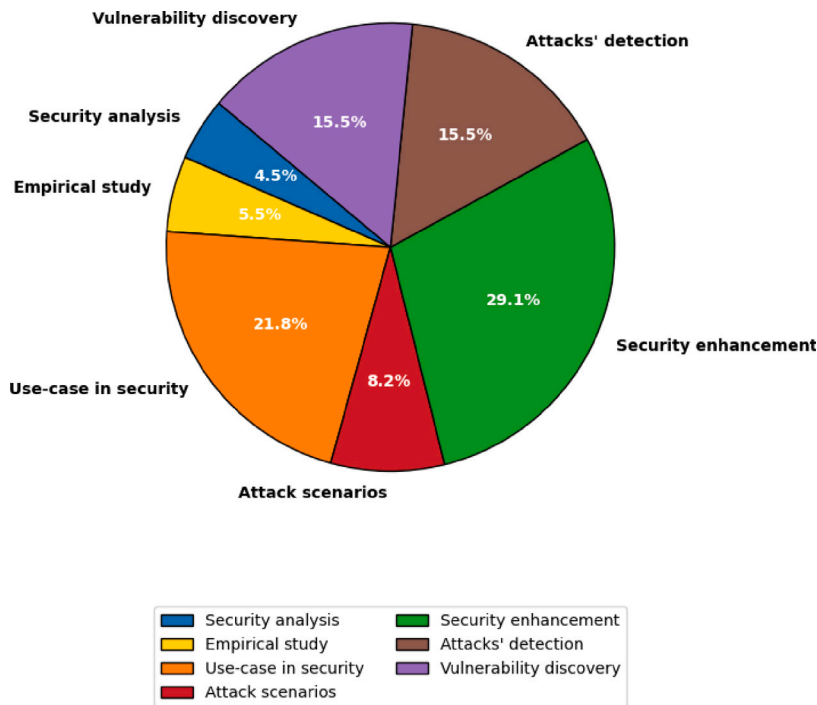


Fig. 1. WebAssembly security research papers categorization.

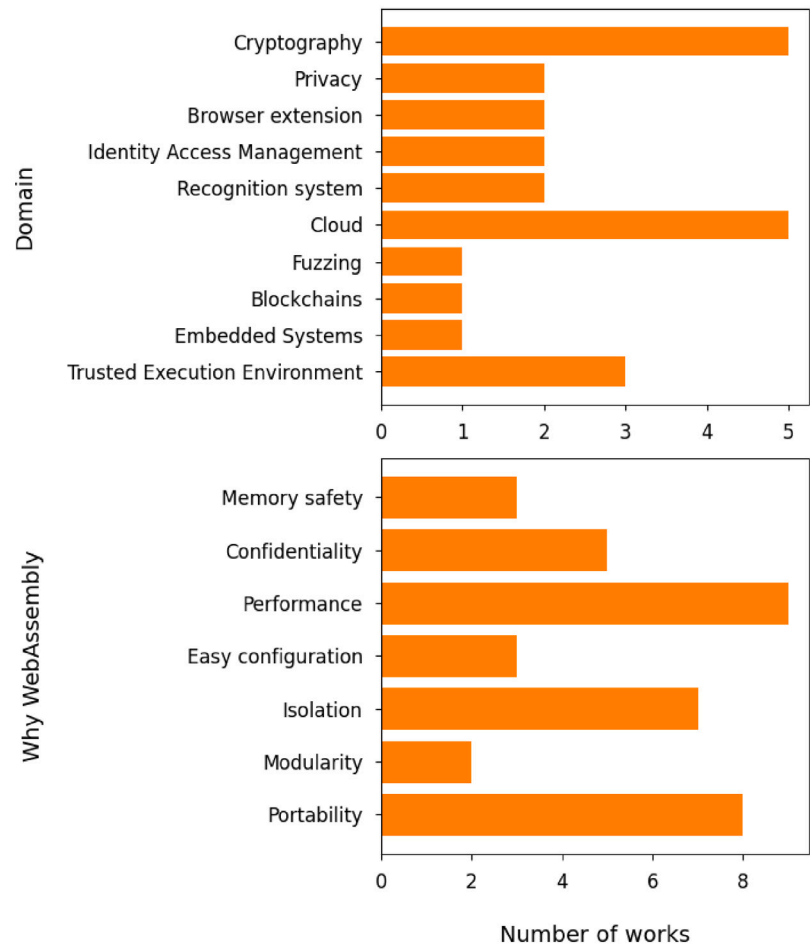


Fig. 2. WebAssembly use-cases.

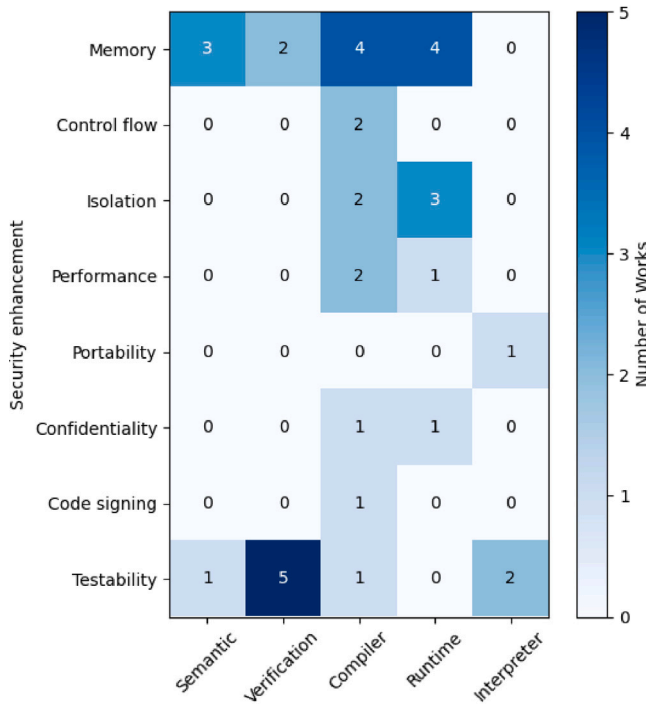


Fig. 3. Heatmap for security enhancements works.

and runtime. As regards testability, it is analyzed through formal verification methods. Indeed, many improvements could be made through the interpreter since it has less impact compared to changes in the runtime and compiler, thus providing a potential direction for future works. Additionally, confidentiality and code signing are other elements that need to be further explored.

6.5. Results availability

For each security category, we analyze whether the authors of relevant works have made available the source code used for the experiments they carried out. The results of this analysis are condensed in Table 12. A detailed list of the publicly available repositories we were able to check is instead reported in Table 14 in Appendix. As it is possible to observe, a significant number of works have publicly released their code. This open-source release of data and code will enable researchers to expand the literature on WebAssembly more rapidly and develop novel approaches in the field. We hope that the collection provided in this work will further boost research advancements.

6.6. Research gaps in WebAssembly and security

Reflecting on the aforementioned considerations, we can identify the following research gaps:

- WebAssembly has been thoroughly analyzed in the real world, especially through the Alexa [69] benchmark, which was retired in 2022. Future studies should investigate additional sources to confirm the empirical results of published works;
- Empirical studies do not address the adoption of WebAssembly on the dark web. Given that this technology is used for malicious activities such as evading malware detection and crypto-mining, exploring the relationship between WebAssembly and the dark web could be a compelling research direction.
- Crypto-mining and smart contracts are highly relevant in WebAssembly research, with different approaches proposed for detecting attacks and discovering vulnerabilities in these fields.

Table 13

Research gaps in WebAssembly Security. Population (P): the group under study. Intervention (I): the action, tool, or technology being investigated. Comparison (C): the alternative to which the intervention is compared. Outcomes (O): the results or changes being measured in the study. Setting (S): the context, or environment where the study is conducted.

PICOS element	Identified gap	Reason(s) for Gap	Proposed research direction
Population (P)	Limited exploration of WebAssembly in dark web environments.	Insufficient information.	Investigate WebAssembly adoption and use in dark web applications for malicious activities such as malware evasion and crypto-mining.
Intervention (I)	Crypto-mining and smart contracts dominate research, limiting the exploration of other applications.	Insufficient or imprecise information.	Study WebAssembly's applications in underexplored fields like AI, cybersecurity training, or performance-critical tasks.
Comparison (C)	Heavy reliance on the Alexa benchmark, retired in 2022, which limits the applicability of findings.	Not the right information.	Develop updated benchmarks and datasets to validate and extend findings of earlier empirical studies.
Outcomes (O)	Security analyses mainly address C programs, neglecting other languages like Rust, Python, and JavaScript.	Insufficient information; not diverse.	Expand security analysis to include WebAssembly binaries compiled from languages other than C, such as Rust and Python.
Setting (S)	Limited focus on practical applications like cyber ranges for security training.	Not the right information; insufficient.	Prototype WebAssembly's use in cyber ranges to enhance practical cybersecurity training scenarios.

However, empirical studies indicate that WebAssembly is also applied to other types of applications. More studies should investigate this relevance and potentially promote research in unexplored fields where WebAssembly is prominently employed.

- WebAssembly is adopted for various security use cases for reasons such as performance, portability, and isolation. However, other interesting applications could be examined. One such application is related to cyber ranges, i.e., realistic offensive and defensive scenarios where people in the security field can be trained [220]. To contribute to this area, we plan to investigate the benefits of using WebAssembly in cyber ranges by comparing our previous research [221–223] to a cyber range solution developed entirely using this technology. One approach we are already pursuing is the use of tools such as *container2wasm*,² which allows for the automated translation between docker images and WebAssembly code that can be run on WASI runtimes. As a related effort, the Docker community has recently released a beta version of a brand new framework called *Wasm workloads*,³ which is an experimental feature allowing docker containers to be run on an ad hoc provided docker runtime and then accessed through the

² <https://github.com/ktock/container2wasm>

³ <https://docs.docker.com/desktop/features/wasm/>

Table 14

Publicly available source codes of analyzed works All the provided links were valid in May 2024. Other works are not included in the analysis.

Security analysis	
Work	Publicly available source
Lehmann et al. (2020) [54]	https://github.com/sola-st/wasm-binary-security
Stiévenart et al. (2022) [56]	https://figshare.com/articles/dataset/SAC_2022_Dataset/17297477
Empirical studies	
Work	Publicly available source
Romano et al. (2021) [72]	https://github.com/sola-st/WasmBench
Hilbig et al. (2021) [18]	https://wasm-compiler-bugs.github.io/
Konoth et al. (2018) [114]	https://github.com/vusec/minesweeper
Tsoupidi et al. (2021) [215]	https://github.com/romits800/Vivienne
Waseem et al. (2024) [74]	https://zenodo.org/records/10528609
Massidda et al. (2024) [67]	https://github.com/manumassi/vuln_wasm
Use case in cybersecurity	
Work	Publicly available source
Narayan et al. (2020) [77]	https://github.com/PLSysSec/rlbox-usenix2020-aec
Izquierdo Riera et al. (2023) [88]	https://github.com/clipaha/clipaha
Goltzsche et al. (2019) [91]	https://github.com/ibr-ds/AccTEE
Ménétrety et al. (2024) [86]	https://github.com/JamesMenetrey/unine-opodis2023
Attrapadung et al. (2018) [81]	https://github.com/herumi/mcl
Baumgärtner et al. (2019) [75]	https://github.com/stg-tud/bp7eval
Attack scenarios	
Work	Publicly available source
Cabrera-Arteaga et al. (2023) [106]	https://github.com/ASSERT-KTH/wasm_evasion
Easdon et al. (2022) [103]	https://github.com/libtea/frameworks
Oz et al. (2023) [110]	https://github.com/cslfiu/Rob_Ransomware_over_Modern_Web_Browsers
Attacks' detection	
Work	Publicly available source
Kharraz et al. (2019) [117]	https://github.com/teamnsrg/outguard
Bian et al. (2020) [118]	https://github.com/cuhk-seclab/MineThrottle
Romano et al. (2020) [121]	https://miner-ray.github.io
Komiya et al. (2024) [127,128]	https://github.com/c-chocolate/jabberwock
Security enhancements	
Work	Publicly available source
Peach et al. (2020) [216]	https://github.com/gwsystems/awasm/
Zhang et al. (2021) [92]	https://github.com/Zhiyi-Zhang/PS-Signature-and-EL-PASSO
Vassena et al. (2021) [161]	https://github.com/PLSysSec/blade
Narayan et al. (2021a) [217]	https://github.com/PLSysSec/swivel
Lei et al. (2023) [218]	https://github.com/PKU-ASAL/PKUWA
Watt et al. (2023) [149]	https://github.com/WasmCert/WasmCert-Isabelle
Menetrey et al. (2021) [64]	https://github.com/jamesmenetrey/unine-twine
Geller et al. (2024) [134]	https://dl.acm.org/doi/10.1145/3580426/full/
Johnson et al. (2023) [219]	https://github.com/PLSysSec/wave
Narayan et al. (2023) [139]	https://github.com/PLSysSec/hfi-root
Watt et al. (2019) [132]	https://github.com/PLSysSec/ct-wasm
Romano et al. (2022) [107]	https://github.com/js2wasm-obfuscator/translator
Jay Bosamiya et al. (2022) [138]	https://github.com/secure-foundations/provably-safe-sandboxing-wasm-usenix22
Michael et al. (2023) [133]	https://github.com/PLSysSec/ms-wasm
Watt et al. (2021) [154]	https://github.com/WasmCert
Menetrey et al. (2022) [214]	https://github.com/JamesMenetrey/unine-watz
Schrammel et al. (2020) [160]	https://github.com/IAIK/Donky
VanHattum et al. (2024) [155]	https://github.com/avanhatt/asplos24-ae-crocus
Gao et al. (2024) [145]	https://github.com/DelphinusLab/zkWasm
Legoupil et al. (2024) [157]	https://zenodo.org/records/13383121

(continued on next page)

browser.

- Almost all security analysis works assess the security of C programs compiled for WebAssembly. However, WebAssembly developers write programs in other languages, such as JavaScript and Python. Additionally, the Rust language is now a memory-safe alternative to C. We suggest expanding security research to explore these languages.

Table 13 provides a more formal overview of found research gaps, through the adoption of the structured framework proposed by Robinson et al. (2011) [224]:

7. Conclusions

In this work, we provide a comprehensive overview of the current state of security research in WebAssembly, highlighting key studies that address various security aspects of this emerging technology. We conducted a thorough review of the most recent literature, analyzing a total of 147 studies, categorizing them into eight distinct categories, and critically evaluating the results.

Our analysis reveals significant research gaps in the WebAssembly security field, which we believe future studies should aim to address. As an example, we identify the lack of exploration of cyber ranges as a

Table 14 (continued).

Vulnerability discovery	
Work	Publicly available source
Khan et al. (2023) [176]	https://github.com/purseclab/FuzzSGX
Li et al. (2022) [166]	https://github.com/lwy0518/datasets_resultsa
Jiang et al. (2021) [171]	https://github.com/gongbell/WANA
Brito et al. (2022) [167]	https://github.com/wasmati/wasmati
Lehmann et al. (2021) [173]	https://github.com/fuzzm/fuzzm-project
Haßler and Maier (2022) [178]	https://github.com/fgsect/WAFL
Quan et al. (2019) [164]	https://github.com/EVulHunter/EVulHunter
Chen et al. (2022) [179]	https://github.com/wasai-project/wasai
Zhou and Chen (2023) [180]	https://github.com/HuskiesUESTC/AntFuzzer-WASMOD
Other works	
Work	Publicly available source
William Fu et al. (2018) [189]	https://github.com/wfus/WebAssembly-Taint
Namjoshi et al. (2021) [202]	https://github.com/nokia/web-assembly-self-certifying-compilation-framework
Narayan et al. (2021) [183]	https://rlbox.dev/
Katherine Hough and Jonathan Bell (2021) [191]	https://doi.org/10.6084/m9.figshare.16611424.v1
Stiévenart et al. (2021,2022) [60,192]	https://github.com/acieroid/wassail
Stiévenart et al. (2023) [193]	https://doi.org/10.5281/zenodo.8157269
Cabrera Arteaga et al. (2021) [61]	https://github.com/ASSERT-KTH/slumps/tree/master/crow
He et al. (2023) [197]	https://github.com/PKU-ASAL/Eunomia-ISSTA23
Stiévenart et al. (2020) [198]	https://github.com/acieroid/wassail/releases/tag/scam2020
Lehmann et al. (2019) [200]	http://wasabi.software-lab.org/
Bastys et al. (2022) [211]	https://github.com/womeier/secwasm
Zhou et al. (2023) [203]	https://github.com/erxiaozhou/WaDiff
Cao et al. (2023) [108]	https://github.com/security-pride/BREWasm
Dejaeghere et al. (2023) [208]	https://gist.github.com/Rekindle2023/ca6a072205698925fa80f928eebe172e
Peng et al. (2023) [212]	https://github.com/rssys/uswitch
Cabrera-Arteaga et al. (2024) [184]	https://github.com/bytecodealliance/wasm-tools/tree/main/crates/wasm-mutate
Zhang et al. (2023) [204]	https://github.com/bnmcxldz/TOSEM2023_Complementary_materials
Zhang et al. (2024) [194]	https://github.com/WasmCFuzz/WasmCFuzz
He et al. (2024) [199]	https://github.com/PKU-ASAL/SeeWasm

^a The work only provides dataset and results, but no source code.

promising yet underutilized security use case in WebAssembly, which has the potential to revolutionize the way we approach security testing and training.

We also highlight the potential for WebAssembly to enable more efficient and effective security testing and discuss the importance of considering the security implications of WebAssembly's unique features, such as its sandboxed execution environment and memory safety guarantees.

We hope that this work will be useful to researchers seeking to make a meaningful impact in the challenging field of enhancing the security features of WebAssembly and provide valuable insights for developers and practitioners looking to leverage WebAssembly for their security applications.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix. Publicly available source code repositories

See Table 14.

Data availability

No data was used for the research described in the article.

References

- [1] Spincorp, Unlocking WebAssembly: The future of high-performance web applications and its industry impact, 2024, <https://www.e-spincorp.com/unlocking-webassembly-future-high-performance-web-applications-industry-impact/>, Online; 2-Dec-2024.
- [2] RobustCloud, Past, present, and future of WebAssembly (wasm) and what it means for the industry, 2023, Online; 2-Dec-2024, <https://www.linkedin.com/pulse/past-present-future-webassembly-wasm-what-means-industry-d9unc>.
- [3] Promwad, Webassembly-in-automotive, 2024, <https://promwad.com/industries/automotive/webassembly-in-automotive>, Online; 27-Nov-2024.
- [4] Otoyá Nakakaze, István Koren, Florian Brillowski, Ralf Klamma, Adaptive retrofitting for industrial machines: utilizing webassembly and peer-to-peer connectivity on the edge, World Wide Web 27 (1) (2024/01/25) 7, <http://dx.doi.org/10.1007/s11280-024-01237-8>,
- [5] codvo.ai, Webassembly-for-high-performing-industrial-and-medical-use-cases, 2022, <https://www.codvo.ai/post/webassembly-for-high-performing-industrial-and-medical-use-cases>, Online; 27-Nov-2024.
- [6] Brendan Eich, From ASM.JS to WebAssembly, 2008, <https://brendaneich.com/2015/06/from-asm-js-to-webassembly/>, Online; 14-Mar-2024.
- [7] Mozilla, Asm.js, 2013, <http://asmjs.org/>, Online; 14-Mar-2024.
- [8] Alliance, Wasm, 2008, <https://github.com/WebAssembly/design/blob/main/FAQ.md>, Online; 14-Mar-2024.
- [9] W3C Consortium, WebAssembly - core, 2008, <https://www.w3.org/TR/wasm-core-1/>, Online; 14-Mar-2024.
- [10] Alliance, Bytecode alliance, 2018, <https://bytecodealliance.org/>, Online; 14-Mar-2024.
- [11] Changyuan Lin, Sarah Nadi, Hamzeh Khazaei, A large-scale data set and an empirical study of docker images hosted on docker hub, in: 2020 IEEE International Conference on Software Maintenance and Evolution, ICSME, 2020, pp. 371–381, <http://dx.doi.org/10.1109/ICSME46990.2020.00043>.
- [12] The Wasmer team, Wasmer, 2018, <https://github.com/wasmerio/wasmer>, Online; 08-Apr-2024.
- [13] The Wasmer team, Wasmer SDK, 2018, <https://github.com/wasmerio/wasmer?#wasmer-sdk>, Online; 08-Apr-2024.
- [14] The Wasmer team, Wasmer edge, 2018, <https://wasmer.io/products/edge>, Online; 08-Apr-2024.

- [15] The Wasmer team, Wasmer hub, 2018, <https://wasmer.io/>, Online; 08-Apr-2024.
- [16] Bytecode Alliance, The state of WebAssembly in 2023, 2019, <https://wasi.dev/>, Online; 08-Apr-2024.
- [17] W3C Consortium, WebAssembly - UseCases, 2008, <https://github.com/WebAssembly/design/blob/main/UseCases.md>, Online; 14-Mar-2024.
- [18] Aaron Hilbig, Daniel Lehmann, Michael Pradel, An empirical study of real-world WebAssembly binaries: Security, languages, use cases, in: Proceedings of the Web Conference 2021, WWW '21, Association for Computing Machinery, New York, NY, USA, ISBN: 9781450383127, 2021, pp. 2696–2708, <http://dx.doi.org/10.1145/3442381.3450138>.
- [19] SlashData (Nikita Solodkov and Brayton Noll) and Cloud Native Computing Foundation, The state of WebAssembly in 2023, 2023, <https://www.cncf.io/wp-content/uploads/2023/09/The-State-of-WebAssembly-2023.pdf>, Online; 08-Apr-2024.
- [20] Colin Eberhardt, The-state-of-weassembly-2023.html, 2023, <https://blog.scottlogic.com/2023/10/18/the-state-of-weassembly-2023.html>, Online; 27-Nov-2024.
- [21] C. Bodei, P. Degano, C. Priami, Checking security policies through an enhanced control flow analysis, *J. Comput. Secur.* 13 (2005) 49–85, <http://dx.doi.org/10.3233/jcs-2005-13103>.
- [22] L. Sampaio, Alessandro F. Garcia, Exploring context-sensitive data flow analysis for early vulnerability detection, *J. Syst. Softw.* 113 (2016) 337–361, <http://dx.doi.org/10.1016/j.jss.2015.12.021>.
- [23] Fabian Yamaguchi, Nico Golde, Daniel Arp, Konrad Rieck, Modeling and discovering vulnerabilities with code property graphs, in: 2014 IEEE Symposium on Security and Privacy, 2014, pp. 590–604, <http://dx.doi.org/10.1109/SP.2014.44>.
- [24] Wesley van der Lee, S. Verwer, Vulnerability detection on mobile applications using state machine inference, in: 2018 IEEE European Symposium on Security and Privacy Workshops, EuroS&P, 2018, pp. 1–10, <http://dx.doi.org/10.1109/EuroSPW.2018.00008>.
- [25] Aseel Alsaedi, Abeer Alhuzali, Omamah Bamasag, Black-box fuzzing approaches to secure web applications: Survey, *Int. J. Adv. Comput. Sci. Appl.* 12 (5) (2021) <http://dx.doi.org/10.14569/IJACSA.2021.0120599>.
- [26] Chen Chen, Baojiang Cui, Jinxin Ma, Runpu Wu, Jianchao Guo, Wenqian Liu, A systematic review of fuzzing techniques, *Comput. Secur.* (ISSN: 0167-4048) 75 (2018) 118–137, <http://dx.doi.org/10.1016/j.cose.2018.02.002>, URL <https://www.sciencedirect.com/science/article/pii/S0167404818300658>.
- [27] P. David Coward, Symbolic execution systems—a review, *Softw. Eng. J.* (ISSN: 0268-6961) 3 (6) (1988) 229, <http://dx.doi.org/10.1049/sej.1988.0029>.
- [28] Arash Sabbaghi, Mohammad Reza Keyvanpour, A systematic review of search strategies in dynamic symbolic execution, *Comput. Stand. Interfaces* (ISSN: 0920-5489) 72 (2020) 103444, <http://dx.doi.org/10.1016/j.csi.2020.103444>, URL <https://www.sciencedirect.com/science/article/pii/S0920548919300066>.
- [29] Lynn M. Foreman, S. Zweben, A study of the effectiveness of control and data flow testing strategies, *J. Syst. Softw.* 21 (1993) 215–228, [http://dx.doi.org/10.1016/0164-1212\(93\)90024-R](http://dx.doi.org/10.1016/0164-1212(93)90024-R).
- [30] Xiaofeng Yang, Mohammad Zulkernine, Secure method calls by instrumenting bytecode with aspects, in: Ehud Gudes, Jaideep Vaidya (Eds.), *Data and Applications Security XXIII*, Springer Berlin Heidelberg, Berlin, Heidelberg, ISBN: 978-3-642-03007-9, 2009, pp. 126–141.
- [31] Yinsheng Li, Emerging blockchain-based applications and techniques, *Serv. Orient. Comput. Appl.* 13 (4) (2019) 279–285, <http://dx.doi.org/10.1007/s11761-019-00281-x>.
- [32] Gavin Wood, Ethereum: A secure decentralised generalised transaction ledger EIP-150 REVISION (759dcd - 2017-08-07), 2017, URL <https://ethereum.github.io/yellowpaper/paper.pdf>. (Accessed: 27 March 2024).
- [33] Ethereum, Ethereum glossary, 2020, <https://ethereum.org/en/glossary>, Online; 14-Mar-2024.
- [34] Ethereum, Introduction to smart contracts, 2021, <https://ethereum.org/en/smart-contracts/>, Online; 14-Mar-2024.
- [35] Ethereum, Introduction to smart contracts, 2021, <https://ethereum.org/en/smart-contracts/>, Online; 14-Mar-2024.
- [36] EOSIO, Eos.io, 2020, <https://eos.io/>, Online; 14-Mar-2024.
- [37] Ningyu He, Haoyu Wang, Lei Wu, Xiapu Luo, Yao Guo, Xiangqun Chen, A survey on EOSIO systems security: Vulnerability, attack, and mitigation, 2022.
- [38] Mohamed Sabt, Mohammed Achemlal, Abdelmadjid Bouabdallah, Trusted execution environment: What it is, and what it is not, in: 2015 IEEE Trustcom/BigDataSE/ISPA, vol. 1, 2015, pp. 57–64, <http://dx.doi.org/10.1109/Trustcom.2015.357>.
- [39] Intel, Software guard extension, 2021, <https://www.intel.com/content/www/us/en/developer/tools/software-guard/extensions/overview.html>, Online; 26-03-2024.
- [40] Jane Webster, Richard T. Watson, Analyzing the past to prepare for the future: Writing a literature review, *MIS Q.* (ISSN: 02767783) 26 (2) (2002) xiii–xxiii, URL <http://www.jstor.org/stable/4132319>.
- [41] Barbara Kitchenham, Pearl Brereton, A systematic review of systematic review process research in software engineering, *Inf. Softw. Technol.* (ISSN: 0950-5849) 55 (12) (2013) 2049–2075, <http://dx.doi.org/10.1016/j.infsof.2013.07.010>, URL <https://www.sciencedirect.com/science/article/pii/S0950584913001560>.
- [42] IEEE xplore, 2000, <https://ieeexplore.ieee.org>. (Accessed: 08 April 2024).
- [43] Elsevier, Scopus, 1996, <https://www.scopus.com/>. Online; 16-Apr-2024.
- [44] 1991, arXiv.org: e-Print archive, <https://arxiv.org>. (Accessed: 08 April 2024).
- [45] Black hat, 1997, <https://www.blackhat.com>. (Accessed: 08 April 2024).
- [46] Philip Kwedza, Stones Dalitso Chindipha, Cryptojacking detection in cloud infrastructure using network traffic, in: 2023 International Conference on Electrical, Computer and Energy Technologies, ICECET, 2023, pp. 1–6, <http://dx.doi.org/10.1109/ICECET58911.2023.10389593>.
- [47] Rui Xiao, Tianyu Li, Soundarya Ramesh, Jun Han, Jinsong Han, MagTracer: Detecting GPU cryptojacking attacks via magnetic leakage signals, in: Proceedings of the 29th Annual International Conference on Mobile Computing and Networking, Association for Computing Machinery, New York, NY, USA, ISBN: 9781450399906, 2023.
- [48] Hamid Darabian, Sajad Homayounoot, Ali Dehghantanha, Sattar Hashemi, Hadis Karimipour, Reza M. Parizi, Kim-Kwang Raymond Choo, Detecting cryptomining malware: a deep learning approach for static and dynamic analysis, *J. Grid Comput.* 18 (2) (2020/06/01) 293–303, <http://dx.doi.org/10.1007/s10723-020-09510-6>.
- [49] Wenwen Wang, How far we've come – a characterization study of standalone WebAssembly runtimes, in: 2022 IEEE International Symposium on Workload Characterization, IISWC, 2022, pp. 228–241, <http://dx.doi.org/10.1109/IISWC55918.2022.00028>.
- [50] Keyan Cao, Yefan Liu, Gongjie Meng, Qimeng Sun, An overview on edge computing research, *IEEE Access* 8 (2020) 85714–85728, <http://dx.doi.org/10.1109/ACCESS.2020.2991734>.
- [51] Sangeeta Kakati, Mats Brorsson, WebAssembly beyond the web: A review for the edge-cloud continuum, in: 2023 3rd International Conference on Intelligent Technologies, CONIT, 2023, pp. 1–8, <http://dx.doi.org/10.1109/CONIT59222.2023.10205816>.
- [52] Detlef Schoder, Introduction to the internet of things, *Internet Things A To Z* (2018) 1–50, <http://dx.doi.org/10.1002/9781119456735.ch1>.
- [53] Partha Pratim Ray, An overview of WebAssembly for IoT: Background, tools, state-of-the-art, challenges, and future directions, *Futur. Internet* 15 (8) (2023) <http://dx.doi.org/10.3390/fi15080275>, Cited by: 0; All Open Access, Gold Open Access.
- [54] Daniel Lehmann, Johannes Kinder, Michael Pradel, Everything old is new again: Binary security of weassembly, in: 29th USENIX Security Symposium (USENIX Security 20), 2020, pp. 217–234, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85091924058&partnerID=40&md5=996f09154d5a3c8001c1c7093dd736c7>, Cited by: 54.
- [55] Quentin Stievenart, Coen De Roover, Mohammad Ghafari, The security risk of lacking compiler protection in WebAssembly, in: IEEE International Conference on Software Quality, Reliability and Security, QRS, 2021-December, 2021, pp. 132–139, <http://dx.doi.org/10.1109/QRS54544.2021.00024>, Cited by: 7; All Open Access, Green Open Access.
- [56] Quentin Stievenart, Coen De Roover, Mohammad Ghafari, Security risks of porting c programs to weassembly, in: Proceedings of the ACM Symposium on Applied Computing, 2022, pp. 1713–1722, <http://dx.doi.org/10.1145/3477314.3507308>, Cited by: 9; All Open Access, Green Open Access.
- [57] Brian McFadden, Tyler Lukasiewicz, Jeff Dileo, Justin Engler, Security chasms of WASM, 2018, <https://i.blackhat.com/us-18/Thu-August-9/us-18-Lukasiewicz-WebAssembly-A-New-World-of-Native-Exploits-On-The-Web-wp.pdf>, Online; 21-Mar-2024.
- [58] Minseo Kim, Hyerean Jang, Youngjoo Shin, Avengers, assemble! survey of WebAssembly security solutions, in: 2022 IEEE 15th International Conference on Cloud Computing, CLOUD, 2022, pp. 543–553, <http://dx.doi.org/10.1109/CLOUD55607.2022.00077>.
- [59] Håkon Harnes, Donn Morrison, SoK: Analysis techniques for WebAssembly, *Futur. Internet* (ISSN: 1999-5903) 16 (3) (2024) 84, <http://dx.doi.org/10.3390/fi16030084>.
- [60] De Roover Stievenart, Wassail: a WebAssembly static analysis library (extended presentation abstract), 2021, <https://drive.google.com/file/d/1BWxJIE57O-o2gBjm-pE7YSTWpSnjEnQy/view>, Online; 21-Mar-2024.
- [61] Javier Cabrera Arteaga, Orestis Floros, Oscar Vera Perez, Benoit Baudry, Martin Monperrus, CROW: Code diversification for WebAssembly, in: Proceedings 2021 Workshop on Measurements, Attacks, and Defenses for the Web, in: MADWeb 2021, Internet Society, 2021, <http://dx.doi.org/10.14722/madweb.2021.23004>.
- [62] Aleph One, Smashing the stack for fun and profit, 1996, *Phrack Magazine*, Vol. 7 no. 49 pages 14–16.
- [63] Brian W. Kernighan, Dennis M. Ritchie, *The C Programming Language*, second ed., Prentice Hall Professional Technical Reference, ISBN: 0131103709, 1988.
- [64] James Menetrey, Marcelo Pasin, Pascal Felber, Valerio Schiavoni, Twine: An embedded trusted runtime for weassembly, in: Proceedings - International Conference on Data Engineering, 2021-April, 2021, pp. 205–216, <http://dx.doi.org/10.1109/ICDE51399.2021.00025>, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85112867383&doi=10.1109%2fICDE51399.2021.00025&partnerID=40&md5=015890b1dd2af1855e44cd3deb61146f>, Cited by: 25; All Open Access, Green Open Access.

- [65] J  mes M  n  tre  y, Marcelo Pasin, Pascal Felber, Valerio Schiavoni, Giovanni Mazzeo, Arne Hollum, Darshan Vaydia, A comprehensive trusted runtime for WebAssembly with intel SGX, *IEEE Trans. Depend. Secur. Comput.* 21 (4) (2024) 3562–3579, <http://dx.doi.org/10.1109/TDSC.2023.3334516>.
- [66] Manuel Niek  , Lennart Almstedt, R  diger Kapitza, Edgedancer: Secure mobile WebAssembly services on the edge, in: *EdgeSys 2021 - Proceedings of the 4th International Workshop on Edge Systems, Analytics and Networking*, Part of EuroSys 2021, 2021, pp. 13–18, <http://dx.doi.org/10.1145/3434770.3459731>, Cited by: 7.
- [67] Emmanuele Massidda, Lorenzo Pisu, Davide Maiorca, Giorgio Giacinto, Bringing binary exploitation at port 80: Understanding C vulnerabilities in WebAssembly, in: *Proceedings of the 21st International Conference on Security and Cryptography, SCITEPRESS - Science and Technology Publications*, 2024, pp. 552–559, <http://dx.doi.org/10.5220/0012852400003767>.
- [68] Jan R  th, Torsten Zimmermann, Konrad Wolsing, Oliver Hohlfeld, Digging into browser-based crypto mining, in: *Proceedings of the Internet Measurement Conference 2018, IMC '18*, Association for Computing Machinery, New York, NY, USA, ISBN: 9781450356190, 2018, pp. 70–76, <http://dx.doi.org/10.1145/3278532.3278539>.
- [69] Peter Dave, Alexa top 1 million websites, 2004, Online; 12-Apr-2024; Alexa service was retired in 2022, Peter Dave provides a GitHub repository as an alternative. <https://github.com/PeterDaveHello/top-1m-domains>.
- [70] Marius Musch, Christian Wressnegger, Martin Johns, Konrad Rieck, New kid on the web: A study on the prevalence of WebAssembly in the wild, in: Roberto Perdici, Cl  mentine Maurice, Giorgio Giacinto, Magnus Almgren (Eds.), *Detection of Intrusions and Malware, and Vulnerability Assessment*, Springer International Publishing, Cham, ISBN: 978-3-030-22038-9, 2019, pp. 23–42.
- [71] Marius Musch, Christian Wressnegger, Martin Johns, Konrad Rieck, New kid on the web: A study on the prevalence of WebAssembly in the wild, in: *Lecture Notes in Computer Science*, Springer International Publishing, (ISSN: 1611-3349) ISBN: 9783030220389, 2019, pp. 23–42, http://dx.doi.org/10.1007/978-3-030-22038-9_2.
- [72] Alan Romano, Xinyue Liu, Yonghui Kwon, Weihang Wang, An empirical study of bugs in WebAssembly compilers, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering, ASE, 2021, pp. 42–54, <http://dx.doi.org/10.1109/ASE51524.2021.9678776>.
- [73] Ivan Puddu, Moritz Schneider, Daniele Lain, Stefano Boschetto, Srdjan   apkun, On (the lack of) code confidentiality in trusted execution environments, in: 2024 IEEE Symposium on Security and Privacy, SP, IEEE, 2024, pp. 4125–4142, <http://dx.doi.org/10.1109/sp54263.2024.00259>.
- [74] Muhammad Waseem, Teerath Das, Aakash Ahmad, Peng Liang, Tommi Mikkonen, Issues and their causes in WebAssembly applications: An empirical study, in: *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, in: EASE 2024, ACM, 2024, pp. 170–180, <http://dx.doi.org/10.1145/3661167.3661227>.
- [75] Lars Baumg  rtner, Jonas H  chst, Tobias Meuser, B-DT7: Browser-based disruption-tolerant networking via bundle protocol 7, in: 6th International Conference on Information and Communication Technologies for Disaster Management, ICT-DM 2019, 2019, <http://dx.doi.org/10.1109/ICT-DM47966.2019.9032944>, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85083227135&doi=10.1109%2fICT-DM47966.2019.9032944&partnerID=40&md5=270bd32a3d57127b5ec195e8eb6b448c>, Cited by: 10.
- [76] Scott Burleigh, Kevin Fall, E. Birrane, Bundle protocol version 7, RFC 9171, RFC Editor, (ISSN: 2070-1721) 2022, URL <https://www.rfc-editor.org/rfc/rfc9171.txt>.
- [77] Shravan Narayan, Craig Disselkoen, Tal Garfinkel, Nathan Froyd, Eric Rahm, Sorin Lerner, Hovav Shacham, Deian Stefan, Retrofitting fine grain isolation in the firefox renderer, in: *Proceedings of the 29th USENIX Security Symposium*, 2020, pp. 699–716, Cited by: 45.
- [78] Ming-Te Chen, Yi Yang Chang, Ta Jen Wu, Digital copyright management mechanism based on dynamic encryption for multiplatform browsers, *Int. J. Semant. Web Inf. Syst.* (ISSN: 1552-6283) 20 (1) (2023) 1–22, <http://dx.doi.org/10.4018/IJSWIS.334591>.
- [79] Daehye Jang, Ammar Askar, Insu Yun, Stephen Tong, Yiqin Cai, Taesoo Kim, Fuzzing@home: Distributed fuzzing on untrusted heterogeneous clients, in: *Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses, RAID '22*, Association for Computing Machinery, New York, NY, USA, ISBN: 9781450397049, 2022, pp. 1–16, <http://dx.doi.org/10.1145/3545948.3545971>.
- [80] Shuzhou Sun, Hui Ma, Zishuai Song, Rui Zhang, WebCloud: Web-based cloud storage for secure data sharing across platforms, *IEEE Trans. Depend. Secur. Comput.* 19 (3) (2022) 1871–1884, <http://dx.doi.org/10.1109/TDSC.2020.3040784>, Cited by: 7.
- [81] Nuttapong Attrapadung, Goichiro Hanaoka, Shigeo Mitsunari, Yusuke Sakai, Kana Shimizu, Tadanori Teruya, Efficient two-level homomorphic encryption in prime-order bilinear groups and a fast implementation in WebAssembly, in: *Proceedings of the 2018 on Asia Conference on Computer and Communications Security*, in: ASIA CCS '18, ACM, 2018, <http://dx.doi.org/10.1145/3196494.3196552>.
- [82] Seog Chung Seo, HeeSeok Kim, Portable and efficient implementation of CRYSTALS-kyber based on WebAssembly, *Comput. Syst. Sci. Eng.* 46 (2) (2023) 2091–2107, <http://dx.doi.org/10.1032604/csse.2023.035064>, Cited by: 1; All Open Access, Hybrid Gold Open Access.
- [83] P. Schwabe, R. Avanzi, J. Bos, L. Ducas, et al., Crystals-kyber, 2021, <https://pq-crystals.org/kyber/index.shtml>. Online; 25-03-2024.
- [84] Shixuan Zhao, Pinshen Xu, Guoxing Chen, Mengya Zhang, Yinqian Zhang, Zhiqiang Lin, Reusable enclaves for confidential serverless computing, in: 32nd USENIX Security Symposium, USENIX Security 2023, vol. 6, 2023, pp. 4015–4032, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85176118235&partnerID=40&md5=2432d064107267e826b266971eb8beb9>, Cited by: 1.
- [85] Lennart Almstedt, Kai Bleeker, Mohammad Mahhouk, Leander Jehl, R  diger Kapitza, Lars Wolf, CONTRACTBOX: Realizing accountable data sharing on the edge using a small scale blockchain, *Comput. Netw.* 229 (2023) <http://dx.doi.org/10.1016/j.comnet.2023.109768>, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85153571862&doi=10.1016%2fj.comnet.2023.109768&partnerID=40&md5=9756e9e5436e81bd535a651e882ee38>, Cited by: 1; All Open Access, Hybrid Gold Open Access.
- [86] J  mes M  n  tre  y, Aeneas Gr  ter, Peterson Yuhala, Julius Oeftiger, Pascal Felber, Marcelo Pasin, Valerio Schiavoni, A Holistic Approach for Trustworthy Distributed Systems with WebAssembly and TEEs, in: Alys  on Bessani, Xavier D  fago, Junya Nakamura, Koichi Wada, Yukiko Yamauchi (Eds.), 27th International Conference on Principles of Distributed Systems (OPODIS 2023), in: *Leibniz International Proceedings in Informatics (LIPIcs)*, vol. 286, Schloss Dagstuhl – Leibniz-Zentrum f  r Informatik, Dagstuhl, Germany, (ISSN: 1868-8969) ISBN: 978-3-95977-308-9, 2024, pp. 23:1–23:23, <http://dx.doi.org/10.4230/LIPIcs.OPODIS.2023.23>, URL <https://drops-dev.dagstuhl.de/entities/document/10.4230/LIPIcs.OPODIS.2023.23>.
- [87] Weizhong Qiang, Zezhao Dong, Hai Jin, Se-lambda: Securing privacy-sensitive serverless applications using sgx enclave, in: *Lecture Notes of the Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering, LNICTS*, vol. 254, 2018, pp. 451–470, http://dx.doi.org/10.1007/978-3-030-01701-9_25, Cited by: 14.
- [88] Francisco Izquierdo Riera, Magnus Almgren, Pablo Picazo-Sanchez, Christian Rohner, Clipaha: A scheme to perform password stretching on the client, in: *Proceedings of the 9th International Conference on Information Systems Security and Privacy - ICISSP, INSTICC, SciTePress*, (ISSN: 2184-4356) ISBN: 978-989-758-624-8, 2023, pp. 58–69, <http://dx.doi.org/10.5220/0011653200003405>.
- [89] Luigi Coppolino, Salvatore D'Antonio, Giovanni Mazzeo, Roberto Nardone, Luigi Romano, Mathieu Schmitt, WASMB  X: A lightweight wasm-based runtime for trustworthy multi-tenant embedded systems, *IEEE Trans. Emerg. Top. Comput.* (ISSN: 2376-4562) (2024) 1–14, <http://dx.doi.org/10.1109/tetc.2024.3409817>.
- [90] Weimin Chen, Xiapu Luo, Haoyu Wang, Heming Cui, Shuyu Zheng, Xuanzhe Liu, EVMBT: A binary translation scheme for upgrading EVM smart contracts to WASM, in: *Proceedings of the 25th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems, LCTES '24*, ACM, 2024, pp. 131–142, <http://dx.doi.org/10.1145/3652032.3657570>.
- [91] David Goltzsche, Manuel Niek  , Thomas Knauth, R  diger Kapitza, AccTEE: A WebAssembly-based two-way sandbox for trusted resource accounting, in: *Proceedings of the 20th International Middleware Conference, Middleware '19*, Association for Computing Machinery, New York, NY, USA, ISBN: 9781450370097, 2019, pp. 123–135, <http://dx.doi.org/10.1145/3361525.3361541>.
- [92] Zhiyi Zhang, Micha   Kr  l, Alberto Sonnino, Lixia Zhang, Etienne Riviere, EL PASSO: Efficient and lightweight privacy-preserving single sign on, *Proc. Priv. Enhanc. Technol.* (ISSN: 2299-0984) 2021 (2) (2021) 70–87, <http://dx.doi.org/10.2478/popets-2021-0018>.
- [93] Ayush Bhardwaj, Navigating the complexities of the cloud-native world: A study of developer perspectives, in: 2023 6th International Conference on Advanced Communication Technologies and Networking (CommNet), 2023, pp. 1–6, <http://dx.doi.org/10.1109/CommNet60167.2023.10365297>.
- [94] J  mes M  n  tre  y, Marcelo Pasin, Pascal Felber, Valerio Schiavoni, WebAssembly as a common layer for the cloud-edge continuum, in: *FRAME 2022 - Proceedings of the 2nd Workshop on Flexible Resource and Application Management on the Edge, Co-located with HPDC 2022*, 2022, pp. 3–8, <http://dx.doi.org/10.1145/3526059.3533618>, Cited by: 12; All Open Access, Green Open Access.
- [95] Danylo Khalyeyev, Tomas Bure  , Petr Hn  tnka, Towards characterization of edge-cloud continuum, in: *Thais Batista, Tom    Bure  , Claudia Raibulet, Henry Muccini (Eds.), Software Architecture. ECSA 2022 Tracks and Workshops*, Springer International Publishing, Cham, ISBN: 978-3-031-36889-9, 2023, pp. 215–230.
- [96] Zishuai Song, Hui Ma, Rui Zhang, Wenhan Xu, Jianhao Li, Everything under control: Secure data sharing mechanism for cloud-edge computing, *IEEE Trans. Inf. Forensics Secur.* 18 (2023) 2234–2249, <http://dx.doi.org/10.1109/TIFS.2023.3266164>.
- [97] Elliott Wen, Gerald Weber, Wasmachine: Bring the edge up to speed with a webassembly OS, in: *IEEE International Conference on Cloud Computing, CLOUD*, 2020-October, 2020, pp. 353–360, <http://dx.doi.org/10.1109/CLOUD49709.2020.00056>, Cited by: 1.

- [98] Shaowen Li, Hiroyuki Sato, W-kernel: An OS kernel architecture designed with isolation and customizability, in: ACM International Conference Proceeding Series, 2023, pp. 42–50, <http://dx.doi.org/10.1145/3637792.3637796>, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85186520460&doi=10.1145%2F3637792.3637796&partnerID=40&md5=b139edd8697176f23f98a54181642188>.
- [99] Rabinbha Karanjai, Weidong Shi, Trusted LLM inference on the edge with smart contracts, in: 2024 IEEE International Conference on Blockchain and Cryptocurrency, ICBC, IEEE, 2024, pp. 1–7, <http://dx.doi.org/10.1109/icbc59979.2024.10634448>.
- [100] Prashaan Pillay, Serestina Viriri, Foresight: Real time facial detection and recognition using webassembly and localized deep neural networks, in: 2019 Conference on Information Communications Technology and Society, ICTAS 2019, 2019, <http://dx.doi.org/10.1109/ICTAS.2019.8703634>, Cited by: 2.
- [101] Ricardo Martín Manso, Pablo Escrivá Gallardo, Face recognition efficiency enhancements using tensorflow and WebAssembly: A practical approach, in: Pablo H. Ruiz, Vanessa Agredo-Delgado, André Luiz Satoshi Kawamoto (Eds.), *Human-Computer Interaction*, Springer International Publishing, Cham, ISBN: 978-3-030-92325-9, 2021, pp. 84–97.
- [102] Daniel Genkin, Lev Pachmanov, Eran Tromer, Yuval Yarom, Drive-by key-extraction cache attacks from portable code, 2018, Cryptology ePrint Archive, Paper 2018/119, URL <https://eprint.iacr.org/2018/119>.
- [103] Catherine Easdon, Michael Schwarz, Martin Schwarzl, Daniel Gruss, Rapid prototyping for microarchitectural attacks, in: 31st USENIX Security Symposium (USENIX Security 22), USENIX Association, Boston, MA, ISBN: 978-1-939133-31-1, 2022, pp. 3861–3877, URL <https://www.usenix.org/conference/usenixsecurity22/presentation/easdon>.
- [104] Thomas Rokicki, Clémentine Maurice, Marina Botvinnik, Yossi Oren, Port contention goes portable: Port contention side channels in web browsers, in: Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security, in: ASIA CCS '22, Association for Computing Machinery, New York, NY, USA, ISBN: 9781450391405, 2022, pp. 1182–1194, <http://dx.doi.org/10.1145/3488932.3517411>.
- [105] Daniel Katzman, William Kosasih, Chitchanok Chuengsatiansup, Eyal Ronen, Yuval Yarom, The gates of time: Improving cache attacks with transient execution, in: 32nd USENIX Security Symposium, USENIX Security 2023, vol. 3, 2023, pp. 1955–1972, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85150326033&partnerID=40&md5=eeca3941ae9d6ff5761b13cf0fa4da33>, Cited by: 3.
- [106] Javier Cabrera-Arteaga, Martin Monperrus, Tim Toady, Benoit Baudry, Webassembly diversification for malware evasion, *Comput. Secur.* (ISSN: 0167-4048) 131 (2023) 103296, <http://dx.doi.org/10.1016/j.cose.2023.103296>, URL <https://www.sciencedirect.com/science/article/pii/S0167404823002067>.
- [107] Alan Romano, Daniel Lehmann, Michael Pradel, Weihang Wang, Wobfuscator: Obfuscating JavaScript malware via opportunistic translation to WebAssembly, in: 2022 IEEE Symposium on Security and Privacy, SP, 2022, pp. 1574–1589, <http://dx.doi.org/10.1109/SP46214.2022.9833626>.
- [108] Shangdong Cao, Ningyu He, Yao Guo, Haoyu Wang, BREWasm: A general static binary rewriting framework for WebAssembly, in: Manuel V. Hermenegildo, José F. Morales (Eds.), *Static Analysis*, Springer Nature Switzerland, Cham, ISBN: 978-3-031-44245-2, 2023, pp. 139–163.
- [109] Nils Loose, Felix Mächtle, Claudius Pott, Volodymyr Bezsmertnyi, Thomas Eisenbarth, Madvex: Instrumentation-based adversarial attacks on machine learning malware detection, in: Daniel Gruss, Federico Maggi, Mathias Fischer, Michele Carminati (Eds.), *Detection of Intrusions and Malware, and Vulnerability Assessment*, Springer Nature Switzerland, Cham, ISBN: 978-3-031-35504-2, 2023, pp. 69–88.
- [110] Harun Oz, Ahmet Aris, Abbas Acar, Güliz Seray Tuncay, Leonardo Babun, Selcuk Uluagac, RoB: Ransomware over modern web browsers, in: 32nd USENIX Security Symposium (USENIX Security 23), USENIX Association, Anaheim, CA, ISBN: 978-1-939133-37-3, 2023, pp. 7073–7090, URL <https://www.usenix.org/conference/usenixsecurity23/presentation/oz>.
- [111] Aditya Tandon, Anand Nayyar, A comprehensive survey on ransomware attack: A growing havoc cyberthreat, in: Valentina Emilia Balas, Neha Sharma, Amlan Chakrabarti (Eds.), *Data Management, Analytics and Innovation*, Springer Singapore, Singapore, ISBN: 978-981-13-1274-8, 2019, pp. 403–420.
- [112] Jo Van Bulck, Marina Minkin, Ofir Weiss, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F. Wenisch, Yuval Yarom, Raoul Strackx, Foreshadow: Extracting the keys to the Intel SGX kingdom with transient out-of-order execution, in: Proceedings of the 27th USENIX Security Symposium, USENIX Association, 2018.
- [113] Jo Van Bulck, Daniel Moghimi, Michael Schwarz, Moritz Lipp, Marina Minkin, Daniel Genkin, Yarom Yuval, Berk Sunar, Daniel Gruss, Frank Piessens, LVI: Hijacking Transient Execution through Microarchitectural Load Value Injection, in: 41th IEEE Symposium on Security and Privacy (S&P'20), 2020.
- [114] Radhesh Krishnan Konoth, Emanuele Vineti, Veelasha Moonsamy, Martina Lindorfer, Christopher Kruegel, Herbert Bos, Giovanni Vigna, MineSweeper: An in-depth look into drive-by cryptocurrency mining and its defense, in: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS '18, Association for Computing Machinery, New York, NY, USA, ISBN: 9781450356930, 2018, pp. 1714–1730, <http://dx.doi.org/10.1145/3243734.3243858>.
- [115] Wenhao Wang, Benjamin Ferrell, Xiaoyang Xu, Kevin W. Hamlen, Shuang Hao, SEISMIC: Secure in-lined script monitors for interrupting cryptojacks, in: Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 11099 LNCS, 2018, pp. 122–142, http://dx.doi.org/10.1007/978-3-319-98989-1_7, Cited by: 44.
- [116] Juan D. Parra Rodriguez, Joachim Posegga, RAPID: Resource and API-based detection against in-browser miners, in: Proceedings of the 34th Annual Computer Security Applications Conference, ACSAC '18, ACM, 2018, <http://dx.doi.org/10.1145/3274694.3274735>.
- [117] Amin Kharraz, Zane Ma, Paul Murley, Charles Lever, Joshua Mason, Andrew Miller, Nikita Borisov, Manos Antonakakis, Michael Bailey, Outguard: Detecting in-browser covert cryptocurrency mining in the wild, in: The World Wide Web Conference, WWW '19, Association for Computing Machinery, New York, NY, USA, ISBN: 9781450366748, 2019, pp. 840–852, <http://dx.doi.org/10.1145/3308558.3313665>, URL <https://doi.org/10.1145/3308558.3313665>.
- [118] Weikang Bian, Wei Meng, Mingxue Zhang, MineThrottle: Defending against wasm in-browser cryptojacking, in: Proceedings of the Web Conference 2020, WWW '20, Association for Computing Machinery, New York, NY, USA, ISBN: 9781450370233, 2020, pp. 3112–3118, <http://dx.doi.org/10.1145/3366423.3380085>.
- [119] Conor Keltan, Aruna Balasubramanian, Ramya Raghavendra, Mudhakar Srivatsa, Browser-based deep behavioral detection of web cryptomining with CoinSpy, in: Proceedings 2020 Workshop on Measurements, Attacks, and Defenses for the Web, in: MADWeb 2020, Internet Society, 2020, <http://dx.doi.org/10.14722/madweb.2020.23002>.
- [120] Mohammad Erfan Mazaheri, Farhad Taheri, Siavash Bayat Sarmadi, Lurking eyes: A method to detect side-channel attacks on JavaScript and WebAssembly, in: 2020 17th International ISC Conference on Information Security and Cryptology, ISCISC, 2020, pp. 1–6, <http://dx.doi.org/10.1109/ISCISC51277.2020.9261920>.
- [121] Alan Romano, Yunhui Zheng, Weihang Wang, MinerRay: Semantics-aware analysis for ever-evolving cryptojacking detection, in: 2020 35th IEEE/ACM International Conference on Automated Software Engineering, ASE, 2020, pp. 1129–1140.
- [122] Guorui Yu, Guangliang Yang, Tongxin Li, Xinhui Han, Shijie Guan, Jialong Zhang, Guofei Gu, MinerGate: A novel generic and accurate defense solution against web based cryptocurrency mining attacks, in: Wei Lu, Qiaoyan Wen, Yuqing Zhang, Bo Lang, Weiping Wen, Hanbing Yan, Chao Li, Li Ding, Ruiguang Li, Yu Zhou (Eds.), *Cyber Security*, Springer Singapore, Singapore, ISBN: 978-981-33-4922-3, 2020, pp. 50–70.
- [123] Faraz Naseem, Ahmet Aris, Leonardo Babun, Ege Tekiner, A. Selcuk Uluagac, MINOS: A lightweight real-time cryptojacking detection system, in: Proceedings 2021 Network and Distributed System Security Symposium, in: NDSS 2021, Internet Society, 2021, <http://dx.doi.org/10.14722/ndss.2021.24444>.
- [124] Franco Tommasi, Christian Catalano, Umberto Corvaglia, Ivan Taurino, MinerAlert: an hybrid approach for web mining detection, *J. Comput. Virol. Hacking Tech.* (ISSN: 2263-8733) 18 (4) (2022) 333–346, <http://dx.doi.org/10.1007/s11416-022-00420-7>.
- [125] Muhammad Haris Khan Abbasi, Subhan Ullah, Tahir Ahmad, Attaullah Buriro, A real-time hybrid approach to combat in-browser cryptojacking malware, *Appl. Sci.* (ISSN: 2076-3417) 13 (4) (2023) 2039, <http://dx.doi.org/10.3390/app13042039>.
- [126] Florian Breitfelder, Tobias Roth, Lars Baumgartner, Mira Mezini, WasmA: A static WebAssembly analysis framework for everyone, in: Proceedings - 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2023, 2023, pp. 753–757, <http://dx.doi.org/10.1109/SANER56733.2023.00085>, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85160520868&doi=10.1109%2FSANER56733.2023.00085&partnerID=40&md5=83da4bf28bbdb65d4e847423509a331>, Cited by: 2.
- [127] Chika Komiya, Naoto Yanai, Kyosuke Yamashita, Shingo Okamura, JABBER-WOCK: A tool for WebAssembly dataset generation and its application to malicious website detection, *J. Inf. Process.* (ISSN: 1882-6652) 32 (2024) 298–307, <http://dx.doi.org/10.2197/ipsjip.32.298>.
- [128] Chika Komiya, Naoto Yanai, Kyosuke Yamashita, Shingo Okamura, JABBER-WOCK: A tool for WebAssembly dataset generation towards malicious website detection, in: 2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W), IEEE, 2023, pp. 36–39, <http://dx.doi.org/10.1109/dsn-w58399.2023.00027>.
- [129] Calebe Helpa, Tiago Heinrich, Marcus Botacin, Newton Will, Rafael Obelheiro, Carlos Maziero, The use of the DWARF debugging format for the identification of potentially unwanted applications (PUAs) in WebAssembly binaries, in: Proceedings of the 21st International Conference on Security and Cryptography, SCITEPRESS - Science and Technology Publications, 2024, pp. 442–449, <http://dx.doi.org/10.5220/0012754500003767>.
- [130] Chris Pickard, Sveta Miladinov, Rogue software: Protection against potentially unwanted applications, in: 2012 7th International Conference on Malicious and Unwanted Software, 2012, pp. 1–8, <http://dx.doi.org/10.1109/MALWARE.2012.6461001>.

- [131] Tiago Heinrich, Newton Will, Rafael Obelheiro, Carlos Maziero, A categorical data approach for anomaly detection in WebAssembly applications, in: Proceedings of the 10th International Conference on Information Systems Security and Privacy, SCITEPRESS - Science and Technology Publications, 2024, pp. 275–284, <http://dx.doi.org/10.5220/0012252800003648>.
- [132] Conrad Watt, John Renner, Natalie Popescu, Sunjay Cauligi, Deian Stefan, CT-WASM: Type-driven secure cryptography for the web ecosystem, Proc. ACM Program. Lang. 3 (POPL) (2019) <http://dx.doi.org/10.1145/3290390>, Cited by: 37; All Open Access, Bronze Open Access, Green Open Access.
- [133] Alexandra E. Michael, Anitha Gollamudi, Jay Bosamiya, Evan Johnson, Aidan Denlinger, Craig Disselkoen, Conrad Watt, Bryan Parno, Marco Patrignani, Marco Vassena, Deian Stefan, MSWasm: Soundly enforcing memory-safe execution of unsafe code, Proc. ACM Program. Lang. 7 (2023) 425–454, <http://dx.doi.org/10.1145/3571208>, Cited by: 2; All Open Access, Bronze Open Access, Green Open Access.
- [134] Adam T. Geller, Justin Frank, William J. Bowman, Indexed types for a statically safe WebAssembly, Proc. ACM Program. Lang. 8 (POPL) (2024) <http://dx.doi.org/10.1145/3632922>.
- [135] Christoph Zenger, Indexed types, Theoret. Comput. Sci. (ISSN: 0304-3975) 187 (1) (1997) 147–165, [http://dx.doi.org/10.1016/S0304-3975\(97\)00062-5](http://dx.doi.org/10.1016/S0304-3975(97)00062-5), URL <https://www.sciencedirect.com/science/article/pii/S0304397597000625>.
- [136] Ziyao Zhang, Wenlong Zheng, Baojian Hua, Qiliang Fan, Zhizhong Pan, Vmcanary: Effective memory protection for WebAssembly via virtual machine-assisted approach, in: IEEE International Conference on Software Quality, Reliability and Security, QRS, 2023, pp. 662–671, <http://dx.doi.org/10.1109/QRS60937.2023.00070>, Cited by: 0.
- [137] Jonathan Protzenko, Benjamin Beurdouche, Denis Merigoux, Karthikeyan Bhargavan, Formally verified cryptographic web applications in webassembly, in: Proceedings - IEEE Symposium on Security and Privacy, Vol. 2019-May, 2019, pp. 1256–1274, <http://dx.doi.org/10.1109/SP.2019.00064>, Cited by: 19; All Open Access, Bronze Open Access, Green Open Access.
- [138] Jay Bosamiya, Wen Shih Lim, Bryan Parno, Provably-Safe multilingual software sandboxing using WebAssembly, in: 31st USENIX Security Symposium (USENIX Security 22), USENIX Association, Boston, MA, ISBN: 978-1-939133-31-1, 2022, pp. 1975–1992, URL <https://www.usenix.org/conference/usenixsecurity22/presentation/bosamiya>.
- [139] Shravan Narayan, Tal Garfinkel, Mohammadkazem Taram, Joey Rudek, Daniel Moghimi, Evan Johnson, Chris Fallin, Anjo Vahldiek-Oberwagner, Michael Lemay, Ravi Sahita, Dean Tullsen, Deian Stefan, Going beyond the limits of SFI: Flexible and secure hardware-assisted in-process isolation with HFI, in: International Conference on Architectural Support for Programming Languages and Operating Systems - ASPLOS, Vol. 3, 2023, pp. 266–281, <http://dx.doi.org/10.1145/3582016.3582023>, Cited by: 2; All Open Access, Bronze Open Access.
- [140] Hosein Yavarzadeh, Mohammadkazem Taram, Shravan Narayan, Deian Stefan, Dean Tullsen, Half&half: Demystifying intel's directional branch predictors for fast, secure partitioned execution, in: 2023 IEEE Symposium on Security and Privacy, SP, 2023, pp. 1220–1237, <http://dx.doi.org/10.1109/SP46215.2023.10179309>.
- [141] Jian Sun, DingYuan Cao, XiMing Liu, ZiYi Zhao, WenWen Wang, XiaoLi Gong, Jin Zhang, SELWasm: A code protection mechanism for WebAssembly, in: 2019 IEEE Intl Conf on Parallel and Distributed Processing with Applications, Big Data and Cloud Computing, Sustainable Computing and Communications, Social Computing and Networking, ISPA/BDCLOUD/SocialCom/SustainCom, 2019, pp. 1099–1106, <http://dx.doi.org/10.1109/ISPA-BDCLOUD-SustainCom-SocialCom48970.2019.00157>.
- [142] Katharina Brandl, Sebastian Erdweg, Sven Keidel, Nils Hansen, Modular abstract definitional interpreters for WebAssembly, in: Leibniz International Proceedings in Informatics, LIPIcs, Vol. 263, 2023, <http://dx.doi.org/10.4230/LIPIcs.ECOOP.2023.5>, Cited by: 0.
- [143] Vasile Adrian Bogdan Pop, Arto Niemi, Valentin Manea, Antti Rusanen, Jan-Erik Ekberg, Towards securely Migrating WebAssembly enclaves, in: EuroSec 2022 - Proceedings of the 15th European Workshop on Systems Security, 2022, pp. 43–49, <http://dx.doi.org/10.1145/3517208.3523755>, Cited by: 0.
- [144] Robin Freyler, Wasmi 0.14.0, 2022, <https://crates.io/crates/wasmi/0.14.0>.
- [145] Sinka Gao, Guoqiang Li, Hongfei Fu, ZKWASM: A ZKSNAK WASM emulator, IEEE Trans. Serv. Comput. (ISSN: 2372-0204) (2024) 1–14, <http://dx.doi.org/10.1109/tsc.2024.3422798>.
- [146] Imam Santoso, Yuli Christyono, Zk-SNARKs as a cryptographic solution for data privacy and security in the digital era, Int. J. Mech. Comput. Manuf. Res. (2023) <http://dx.doi.org/10.35335/computational.v12i2.122>.
- [147] Conrad Watt, Mechanising and verifying the WebAssembly specification, in: Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP '18, ACM, 2018, <http://dx.doi.org/10.1145/3167082>.
- [148] Lawrence C. Paulson, The foundation of a generic theorem prover, J. Automat. Reason. 5 (3) (1989) 363–397, <http://dx.doi.org/10.1007/BF00248324>.
- [149] Conrad Watt, Maja Trela, Peter Lammich, Florian Märkl, WasmRef-isabelle: A verified monadic interpreter and industrial fuzzing oracle for WebAssembly, Proc. ACM Program. Lang. 7 (PLDI) (2023) <http://dx.doi.org/10.1145/3591224>.
- [150] Tobias Nipkow, Lawrence C. Paulson, Markus Wenzel, Isabelle/HOL: a proof assistant for higher-order logic, vol. 2283, Springer Science & Business Media, 2002.
- [151] bytecodealliance, Wasmtime, 2016, <https://github.com/bytecodealliance/wasmtime>, Online; 18-Apr-2024.
- [152] Evan Johnson, David Thien, Yousef Alhessi, Shravan Narayan, Fraser Brown, Sorin Lerner, Tyler McMullen, Stefan Savage, Deian Stefan, SFI safety for native-compiled wasm, in: Network and Distributed Systems Security (NDSS) Symposium, 2021, <http://dx.doi.org/10.14722/ndss.2021.24078>.
- [153] Robert Wahbe, Steven Lucco, Thomas E. Anderson, Susan L. Graham, Efficient software-based fault isolation, SIGOPS Oper. Syst. Rev. (ISSN: 0163-5980) 27 (5) (1993) 203–216, <http://dx.doi.org/10.1145/173668.168635>.
- [154] Conrad Watt, Xiaojia Rao, Jean Pichon-Pharabod, Martin Bodin, Philippa Gardner, Two mechanisations of WebAssembly 1.0, in: FM 2021 - Formal Methods, Beijing, China, 2021, pp. 1–19, URL <https://hal.science/hal-03353748>.
- [155] Alexa VanHattum, Monica Pardeshi, Chris Fallin, Adrian Sampson, Fraser Brown, Lightweight, modular verification for WebAssembly-to-native instruction selection, in: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1, ASPLOS '24, ACM, 2024, pp. 231–248, <http://dx.doi.org/10.1145/3617232.3624862>.
- [156] Bytecode Alliance, Cranelift, 2023, <https://cranelift.dev/>, Online; 14-Mar-2024.
- [157] Maxime Legoupil, June Rousseau, Aïna Linn Georges, Jean Pichon-Pharabod, Lars Birkedal, Iris-MSWasm: Elucidating and mechanising the security invariants of memory-safe WebAssembly, Proc. ACM Program. Lang. (ISSN: 2475-1421) 8 (OOPSLA2) (2024) 304–332, <http://dx.doi.org/10.1145/3689722>.
- [158] Yifan Xia, Ping He, Xuhong Zhang, Peiyu Linn, Shouling Ji, Wenhai Wang, Static semantics reconstruction for enhancing JavaScript-WebAssembly multilingual malware detection, in: Gene Tsudik, Mauro Conti, Kaitai Liang, Georgios Smaragdakis (Eds.), Computer Security - ESORICS 2023, Springer Nature Switzerland, Cham, ISBN: 978-3-031-51476-0, 2024, pp. 255–276.
- [159] Hanxian Huang, Jishen Zhao, Multi-modal learning for WebAssembly reverse engineering, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISTA '24, ACM, 2024, pp. 453–465, <http://dx.doi.org/10.1145/3650212.3652141>.
- [160] David Schrammel, Samuel Weiser, Stefan Steinegger, Martin Schwarzl, Michael Schwarz, Stefan Mangard, Daniel Gruss, Donky: Domain keys – efficient In-Process isolation for RISC-V and x86, in: 29th USENIX Security Symposium (USENIX Security 20), USENIX Association, ISBN: 978-1-939133-17-5, 2020, pp. 1677–1694, URL <https://www.usenix.org/conference/usenixsecurity20/presentation/schrammel>.
- [161] Marco Vassena, Craig Disselkoen, Klaus von Gleissenthall, Sunjay Cauligi, Rami Gökhan Kıcı, Ranjit Jhala, Dean Tullsen, Deian Stefan, Automatically eliminating speculative leaks from cryptographic code with blade, Proc. ACM Program. Lang. 5 (POPL) (2021) <http://dx.doi.org/10.1145/3434330>.
- [162] Suhyeon Song, Seonghwan Park, Donghyun Kwon, metaSafer: A technique to detect heap metadata corruption in WebAssembly, IEEE Access 11 (2023) 124887–124898, <http://dx.doi.org/10.1109/ACCESS.2023.3327817>.
- [163] Shravan Narayan, Tal Garfinkel, Mohammadkazem Taram, Joey Rudek, Daniel Moghimi, Evan Johnson, Chris Fallin, Anjo Vahldiek-Oberwagner, Michael LeMay, Ravi Sahita, Dean Tullsen, Deian Stefan, Hardware-assisted fault isolation: Going beyond the limits of software-based sandboxing, IEEE Micro (ISSN: 1937-4143) 44 (4) (2024) 70–79, <http://dx.doi.org/10.1109/mm.2024.3422977>.
- [164] Lijin Quan, Lei Wu, Haoyu Wang, EVulHunter: Detecting fake transfer vulnerabilities for eosio's smart contracts at webassembly-level, 2019.
- [165] Zhiqiang Yang, Han Liu, Yue Li, Huixuan Zheng, Lei Wang, Bangdao Chen, Seraph: Enabling cross-platform security analysis for EVM and wasm smart contracts, in: Proceedings - International Conference on Software Engineering, 2020, pp. 21–24, <http://dx.doi.org/10.1145/3377812.3382157>, Cited by: 8.
- [166] Wenyuan Li, Jiahao He, Gansen Zhao, Jinji Yang, Shuangyin Li, Ruilin Lai, Ping Li, Hua Tang, Haoyu Luo, Ziheng Zhou, Eosioanalyzer: An effective static analysis vulnerability detection framework for EOSIO smart contracts, in: Proceedings - 2022 IEEE 46th Annual Computers, Software, and Applications Conference, COMPSAC 2022, 2022, pp. 746–756, <http://dx.doi.org/10.1109/COMPSAC54236.2022.00124>, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85136912302&doi=10.1109%2fCOMPSAC54236.2022.00124&partnerID=40&md5=c8cea9b73a0310db1da6e5acadb1bb15>, Cited by: 1.
- [167] Tiago Brito, Pedro Lopes, Nuno Santos, José Frago Santos, Wasmati: An efficient static vulnerability scanner for WebAssembly, Comput. Secur. 118 (2022) <http://dx.doi.org/10.1016/j.cose.2022.102745>, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85130090453&doi=10.1016%2fj.cose.2022.102745&partnerID=40&md5=08c63bf48f2c9f9f3a4cc09bdcccd7dbf>, Cited by: 7; All Open Access, Green Open Access, Hybrid Gold Open Access.
- [168] Markus Scherer, Jeppe Fredsgaard Blaabjerg, Alexander Sjösten, Magdalena Solitro, Matteo Maffei, Wappler: Sound reachability analysis for WebAssembly, in: 2024 IEEE 37th Computer Security Foundations Symposium, CSF, IEEE, 2024, pp. 249–264, <http://dx.doi.org/10.1109/csf61375.2024.00025>.

- [169] Caichang Tu, Wansen Wang, Kaiwen Shi, Yan Xiong, Wenchao Huang, EOSVERIF: An automated analyzer for EOSIO smart contracts, in: Proceedings - 2023 9th International Conference on Big Data Computing and Communications, BigCom 2023, 2023, pp. 103–109, <http://dx.doi.org/10.1109/BIGCOM61073.2023.00022>, Cited by: 0.
- [170] Yuhe Huang, Bo Jiang, W.K. Chan, EOSFuzzer: Fuzzing EOSIO smart contracts for vulnerability detection, in: Proceedings of the 12th Asia-Pacific Symposium on Internetware, Internetwork '20, Association for Computing Machinery, New York, NY, USA, ISBN: 9781450388191, 2021, pp. 99–109, <http://dx.doi.org/10.1145/3457913.3457920>.
- [171] Bo Jiang, Yifei Chen, Dong Wang, Imran Ashraf, W.K. Chan, WANA: Symbolic execution of wasm bytecode for extensible smart contract vulnerability detection, in: IEEE International Conference on Software Quality, Reliability and Security, QRS, 2021-December, 2021, pp. 926–937, <http://dx.doi.org/10.1109/QRS54544.2021.00102>, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85129469733&doi=10.1109%2fQRS54544.2021.00102&partnerID=40&md5=d25d0afb68b861118e0c1edf4173fb8>, Cited by: 9.
- [172] Ningyu He, Ruiyi Zhang, Haoyu Wang, Lei Wu, Xiapu Luo, Yao Guo, Ting Yu, Xuxian Jiang, EOSAFE: Security analysis of EOSIO smart contracts, in: Proceedings of the 30th USENIX Security Symposium, 2021, pp. 1271–1288, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85107913778&partnerID=40&md5=de85f126adfcdf876acab07482d650d>, Cited by: 33.
- [173] Daniel Lehmann, Martin Toldam Torp, Michael Pradel, Fuzzm: Finding memory bugs through binary-only instrumentation and fuzzing of WebAssembly, 2021.
- [174] Filipe Marques, José Frago Santos, Nuno Santos, Pedro Adão, Concolic Execution for WebAssembly, in: Karim Ali, Jan Vitek (Eds.), 36th European Conference on Object-Oriented Programming (ECOOP 2022), in: Leibniz International Proceedings in Informatics (LIPIcs), vol. 222, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, (ISSN: 1868-8969) ISBN: 978-3-95977-225-9, 2022, pp. 11:1–11:29, <http://dx.doi.org/10.4230/LIPIcs.ECOOP.2022.11>, URL <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2022.11>.
- [175] Shuo Yang, Huizhong Li, Zibin Zheng, WaLi: Control-flow-based analysis of wasm smart contracts, Commun. Comput. Inf. Sci. 1679 CCIS (2022) 322–335, http://dx.doi.org/10.1007/978-981-19-8043-5_23, Cited by: 0.
- [176] Arslan Khan, Muqi Zou, Kyungtae Kim, Dongyan Xu, Antonio Bianchi, Dave Jing Tian, Fuzzing SGX enclaves via host program mutations, in: 2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P), 2023, pp. 472–488, <http://dx.doi.org/10.1109/EuroSP57164.2023.00035>.
- [177] Ling Jin, Yinzhi Cao, Yan Chen, Di Zhang, Simone Campanoni, Exgen: Cross-platform, automated exploit generation for smart contract vulnerabilities, IEEE Trans. Depend. Secur. Comput. 20 (1) (2023) 650–664, <http://dx.doi.org/10.1109/TDSC.2022.3141396>.
- [178] Keno Haßler, Dominik Maier, WAFI: Binary-only WebAssembly fuzzing with fast snapshots, in: Reversing and Offensive-Oriented Trends Symposium, ROOTS '21, Association for Computing Machinery, New York, NY, USA, ISBN: 9781450396028, 2022, pp. 23–30, <http://dx.doi.org/10.1145/3503921.3503924>, URL <https://doi.org/10.1145/3503921.3503924>.
- [179] Weimin Chen, Zihan Sun, Haoyu Wang, Xiapu Luo, Haipeng Cai, Lei Wu, WASAI: Uncovering vulnerabilities in wasm smart contracts, in: ISSTA 2022 - Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 703–715, <http://dx.doi.org/10.1145/3533767.3534218>, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85136829573&doi=10.1145%2f3533767.3534218&partnerID=40&md5=02d4bde4b4c9626e4927b48da5eb2bd1>, Cited by: 7; All Open Access, Bronze Open Access.
- [180] Jianfei Zhou, Ting Chen, WASMOD: Detecting vulnerabilities in wasm smart contracts, IET Blockchain 3 (2023) <http://dx.doi.org/10.1049/blc2.12029>.
- [181] Bing Zhang, Jingyue Li, Jiadong Ren, Guoyan Huang, Efficiency and effectiveness of web application vulnerability detection approaches: A review, ACM Comput. Surv. (ISSN: 0360-0300) 54 (9) (2021) <http://dx.doi.org/10.1145/3474553>.
- [182] Marco Abbadini, Michele Beretta, Dario Facchinetti, Gianluca Oldani, Matthew Rossi, Stefano Paraboschi, POSTER: Leveraging eBPF to enhance sandboxing of WebAssembly runtimes, in: Proceedings of the ACM Conference on Computer and Communications Security, 2023, pp. 1028–1030, <http://dx.doi.org/10.1145/3579856.3592831>, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85168163646&doi=10.1145%2f3579856.3592831&partnerID=40&md5=ad1d8a658837b684ca8bf5ab8e106c05>, Cited by: 2; All Open Access, Bronze Open Access.
- [183] Shravan Narayan, Craig Disselkoen, Deian Stefan, Tutorial: Sandboxing (unsafe) c code with rlbbox, in: 2021 IEEE Secure Development Conference (SecDev), 2021, pp. 11–12, <http://dx.doi.org/10.1109/SecDev51306.2021.00017>.
- [184] Javier Cabrera-Arteaga, Nicholas Fitzgerald, Martin Monperrus, Benoit Baudry, WASM-MUTATE: Fast and effective binary diversification for WebAssembly, Comput. Secur. 139 (2024) <http://dx.doi.org/10.1016/j.cose.2024.103731>, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85183204402&doi=10.1016%2fj.cose.2024.103731&partnerID=40&md5=787b8d0138d3d4b0c1366cd98c1584d8>, Cited by: 0; All Open Access, Green Open Access, Hybrid Gold Open Access.
- [185] Shrenik Bhansali, Ahmet Aris, Abbas Acar, Harun Oz, A. Selcuk Ulugac, A first look at code obfuscation for WebAssembly, in: Proceedings of the 15th ACM Conference on Security and Privacy in Wireless and Mobile Networks, WiSec '22, Association for Computing Machinery, New York, NY, USA, ISBN: 9781450392167, 2022, pp. 140–145, <http://dx.doi.org/10.1145/3507657.3528560>.
- [186] Jeongpil Park, Eun-Sun Cho, WebMO: WebAssembly memory bitwise operation obfuscation, in: 2024 IEEE 48th Annual Computers, Software, and Applications Conference, COMPSAC, IEEE, 2024, pp. 1436–1439, <http://dx.doi.org/10.1109/compsac61105.2024.00189>.
- [187] Wenlong Zheng, Baojian Hua, Zhuochen Jiang, WASMDYPA: Effectively detecting WebAssembly bugs via dynamic program analysis, in: Proceedings - 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security Companion, QRS-C 2023, 2023, pp. 867–868, <http://dx.doi.org/10.1109/QRS-C60940.2023.00101>, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85186746180&doi=10.1109%2fQRS-C60940.2023.00101&partnerID=40&md5=e5cdace21e5b6e08428c3ef754c9c16>.
- [188] Edoardo Tinto, Tullio Vardanega, A runtime infrastructure for the continuum of computing, in: Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing, HPDC '24, ACM, 2024, pp. 401–404, <http://dx.doi.org/10.1145/3625549.3658832>.
- [189] William Fu, Raymond Lin, Daniel Inge, TaintAssembly: Taint-based information flow control tracking for WebAssembly, 2018.
- [190] Aron Szanto, Timothy Tamm, Artidoro Pagnoni, Taint tracking for WebAssembly, 2018, arXiv:1807.08349.
- [191] Katherine Hough, Jonathan Bell, A practical approach for dynamic taint tracking with control-flow relationships, ACM Trans. Softw. Eng. Methodol. (TOSEM) 31 (2021) 1–43, <http://dx.doi.org/10.1145/3485464>.
- [192] Quentin Stievenart, David W. Binkley, Coen De Roover, Static stack-preserving intra-procedural slicing of WebAssembly binaries, in: 2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings, Vol. 2022-May, ICSE-Companion, 2022, pp. 2031–2042, <http://dx.doi.org/10.1145/3510003.3510070>, Cited by: 12.
- [193] Quentin Stievenart, David Binkley, Coen De Roover, Dynamic slicing of WebAssembly binaries, in: 2023 IEEE International Conference on Software Maintenance and Evolution, ICSME, 2023, pp. 84–96, <http://dx.doi.org/10.1109/ICSME58846.2023.00020>.
- [194] Xiangwei Zhang, Junjie Wang, Xiaoning Du, Shuang Liu, WasmCFuzz: Structure-aware fuzzing for wasm compilers, in: Proceedings of the 2024 ACM/IEEE 4th International Workshop on Engineering and Cybersecurity of Critical Systems (EnCyCris) and 2024 IEEE/ACM Second International Workshop on Software Vulnerability, in: EnCyCris/SVM '24, ACM, 2024, pp. 1–5, <http://dx.doi.org/10.1145/3643662.3643959>.
- [195] Jideng Han, Zhaoxin Zhang, Yuejin Du, Wei Wang, Xiuyuan Chen, Esfuzzer: An efficient way to fuzz WebAssembly interpreter, Electronics (ISSN: 2079-9292) 13 (8) (2024) 1498, <http://dx.doi.org/10.3390/electronics13081498>.
- [196] Conrad Watt, Petar Maksimović, Neelakantan R. Krishnaswami, Philippa Gardner, A program logic for first-order encapsulated WebAssembly, in: 33rd European Conference on Object-Oriented Programming (ECOOP 2019), Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, <http://dx.doi.org/10.4230/LIPIcs.ECOOP.2019.9>, URL <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2019.9>.
- [197] Ningyu He, Zhehao Zhao, Jikai Wang, Yubin Hu, Shengjian Guo, Haoyu Wang, Guangtai Liang, Ding Li, Xiangqun Chen, Yao Guo, Eunomia: Enabling user-specified fine-grained search in symbolically executing WebAssembly binaries, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, in: ISSTA 2023, Association for Computing Machinery, New York, NY, USA, ISBN: 9798400702211, 2023, pp. 385–397, <http://dx.doi.org/10.1145/3597926.3598064>.
- [198] Quentin Stievenart, Coen De Roover, Compositional information flow analysis for WebAssembly programs, in: Proceedings - 20th IEEE International Working Conference on Source Code Analysis and Manipulation, SCAM 2020, 2020, pp. 13–24, <http://dx.doi.org/10.1109/SCAM51674.2020.00007>, Cited by: 20; All Open Access, Green Open Access.
- [199] Ningyu He, Zhehao Zhao, Hanqin Guan, Jikai Wang, Shuo Peng, Ding Li, Haoyu Wang, Xiangqun Chen, Yao Guo, SeeWasm: An efficient and fully-functional symbolic execution engine for WebAssembly binaries, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA '24, ACM, 2024, pp. 1816–1820, <http://dx.doi.org/10.1145/3650212.3685300>.
- [200] Daniel Lehmann, Michael Pradel, Wasabi: A framework for dynamically analyzing WebAssembly, in: International Conference on Architectural Support for Programming Languages and Operating Systems - ASPLOS, 2019, pp. 1045–1058, <http://dx.doi.org/10.1145/3297858.3304068>, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85064658571&doi=10.1145%2f3297858.3304068&partnerID=40&md5=59ae4019c5efc002e544c0a2806d162e>, Cited by: 44.

- [201] Igor Kottenko, Konstantin Izrailov, Mikhail Buinevich, Igor Saenko, Rajeev Shorey, Modeling the development of energy network software, taking into account the detection and elimination of vulnerabilities, *Energies* (ISSN: 1996-1073) 16 (13) (2023) 5111, <http://dx.doi.org/10.3390/en16135111>.
- [202] Kedar S. Namjoshi, Anton Xue, A self-certifying compilation framework for WebAssembly, in: Fritz Henglein, Sharon Shoham, Yakir Vizel (Eds.), *Verification, Model Checking, and Abstract Interpretation*, Springer International Publishing, Cham, ISBN: 978-3-030-67067-2, 2021, pp. 127–148.
- [203] Shiyao Zhou, Muhui Jiang, Weimin Chen, Hao Zhou, Haoyu Wang, Xiapu Luo, WADIFF: A differential testing framework for WebAssembly runtimes, in: 2023 38th IEEE/ACM International Conference on Automated Software Engineering, ASE, 2023, pp. 939–950, <http://dx.doi.org/10.1109/ASE56229.2023.00188>.
- [204] Yixuan Zhang, Shangdong Cao, Haoyu Wang, Zhenpeng Chen, Xiapu Luo, Dongliang Mu, Yun Ma, Gang Huang, Xuanzhe Liu, Characterizing and detecting WebAssembly runtime bugs, *ACM Trans. Softw. Eng. Methodol.* (ISSN: 1557-7392) 33 (2) (2023) 1–29, <http://dx.doi.org/10.1145/3624743>.
- [205] Munawar Hafiz, Ming Fang, Game of detections: how are security vulnerabilities discovered in the wild? *Empir. Softw. Eng.* 21 (5) (2016) 1920–1959, <http://dx.doi.org/10.1007/s10664-015-9403-7>.
- [206] Tom Lauwaerts, Carlos Rojas Castillo, Robbert Gurdeep Singh, Matteo Marra, Christophe Scholliers, Elisa Gonzalez Boix, Event-based out-of-place debugging, in: *Proceedings of the 19th International Conference on Managed Programming Languages and Runtimes, MPLR '22*, Association for Computing Machinery, New York, NY, USA, ISBN: 9781450396967, 2022, pp. 85–97, <http://dx.doi.org/10.1145/3546918.3546920>.
- [207] Tom Lauwaerts, Robbert Gurdeep Singh, Christophe Scholliers, Warduino: An embedded WebAssembly virtual machine, *J. Comput. Lang.* (ISSN: 2590-1184) 79 (2024) 101268, <http://dx.doi.org/10.1016/j.cola.2024.101268>, URL <https://www.sciencedirect.com/science/article/pii/S259011842400011X>.
- [208] Jules Dejaeghere, Bolaji Gbadamosi, Tobias Pulls, Florentin Rochet, Comparing security in eBPF and WebAssembly, in: *EBPF 2023 - Proceedings of the ACM SIGCOMM 2023 Workshop on EBPF and Kernel Extensions*, 2023, pp. 35–41, <http://dx.doi.org/10.1145/3609021.3609306>, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85169019212&doi=10.1145%2f3609021.3609306&partnerID=40&md5=47b82334ddc5d0a4a4061cbacc5fb88f>, Cited by: 1; All Open Access, Hybrid Gold Open Access.
- [209] Elazar Gershuni, Nadav Amit, A. Gurfinkel, Nina Narodytska, J. Navas, N. Rinetzy, L. Ryzhyk, Shmuel Sagiv, Simple and precise static analysis of untrusted linux kernel extensions, in: *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2019, <http://dx.doi.org/10.1145/3314221.3314590>.
- [210] Steven Pham, Kaue Oliveira, Chung-Hong Lung, WebAssembly modules as alternative to docker containers in IoT application development, in: 2023 IEEE 3rd International Conference on Electronic Communications, Internet of Things and Big Data, ICEIB 2023, 2023, pp. 519–524, <http://dx.doi.org/10.1109/ICEIB57887.2023.10170519>, Cited by: 1.
- [211] Iulia Bastys, Maximilian Algehed, Alexander Sjösten, Andrei Sabelfeld, SecWasm: Information flow control for WebAssembly, in: *Static Analysis: 29th International Symposium, SAS 2022, Auckland, New Zealand, December 5–7, 2022*, Proceedings, Springer-Verlag, Berlin, Heidelberg, ISBN: 978-3-031-22307-5, 2022, pp. 74–103, http://dx.doi.org/10.1007/978-3-031-22308-2_5.
- [212] Dinglan Peng, Congyu Liu, Tapti Palit, Pedro Fonseca, Anjo Vahldiek-Oberwagner, Mona Vij, Uswitch: Fast kernel context isolation with implicit context switches, in: 2023 IEEE Symposium on Security and Privacy, SP, 2023, pp. 2956–2973, <http://dx.doi.org/10.1109/SP46215.2023.10179284>.
- [213] Tong Sun, Borui Li, Yixiao Teng, Yi Gao, Wei Dong, dTEE: A declarative approach to secure IoT applications using TrustZone, in: 2024 23rd ACM/IEEE International Conference on Information Processing in Sensor Networks, IPSN, 2024, pp. 200–212, <http://dx.doi.org/10.1109/IPSIN61024.2024.00021>.
- [214] James Menetrey, Marcelo Pasin, Pascal Felber, Valerio Schiavoni, Watz: A trusted WebAssembly runtime environment with remote attestation for TrustZone, in: *Proceedings - International Conference on Distributed Computing Systems*, Vol. 2022-July, 2022, pp. 1177–1189, <http://dx.doi.org/10.1109/ICDCS54860.2022.00116>, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85136199855&doi=10.1109%2fICDCS54860.2022.00116&partnerID=40&md5=bfd9eefc61b9f32981bdcfeef6746269>, Cited by: 10; All Open Access, Green Open Access.
- [215] Rodothea Myersini Tsoupidi, Musard Balliu, Benoit Baudry, Vivienne: Relational verification of cryptographic implementations in WebAssembly, in: 2021 IEEE Secure Development Conference, SecDev, 2021, pp. 94–102, <http://dx.doi.org/10.1109/SecDev51306.2021.00029>.
- [216] Gregor Peach, Runyu Pan, Zhuoyi Wu, Gabriel Parmer, Christopher Haster, Ludmila Cherkasova, EWASM: Practical software fault isolation for reliable embedded devices, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 39 (11) (2020) 3492–3505, <http://dx.doi.org/10.1109/TCAD.2020.3012647>, Cited by: 12.
- [217] Shravan Narayan, Craig Disselkoen, Daniel Moghimi, Sunjay Cauligi, Evan Johnson, Zhao Gang, Anjo Vahldiek-Oberwagner, Ravi Sahita, Hovav Shacham, Dean Tullsen, Deian Stefan, Swivel: Hardening WebAssembly against spectre, in: 30th USENIX Security Symposium (USENIX Security 21), USENIX Association, ISBN: 978-1-939133-24-3, 2021, pp. 1433–1450, URL <https://www.usenix.org/conference/usenixsecurity21/presentation/narayan>.
- [218] Hanwen Lei, Ziqi Zhang, Shaokun Zhang, Peng Jiang, Zhineng Zhong, Ningyu He, Ding Li, Yao Guo, Xiangqun Chen, Put your memory in order: Efficient domain-based memory isolation for WASM applications, in: *CCS 2023 - Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 904–918, <http://dx.doi.org/10.1145/3576915.3623205>, Cited by: 0.
- [219] Evan Johnson, Evan Laufer, Zijie Zhao, Dan Gohman, Shravan Narayan, Stefan Savage, Deian Stefan, Fraser Brown, Wave: a verifiably secure WebAssembly sandboxing runtime, in: 2023 IEEE Symposium on Security and Privacy, SP, 2023, pp. 2940–2955, <http://dx.doi.org/10.1109/SP46215.2023.10179357>.
- [220] M. Yamin, Basel Katt, V. Gkioulos, Cyber ranges and security testbeds: Scenarios, functions, tools and architecture, *Comput. Secur.* 88 (2020) <http://dx.doi.org/10.1016/j.cose.2019.101636>.
- [221] G. Perrone, S.P. Romano, The docker security playground: A hands-on approach to the study of network security, in: 2017 Principles, Systems and Applications of IP Telecommunications (IPTComm), 2017, pp. 1–8, <http://dx.doi.org/10.1109/IPTCOMM.2017.8169747>.
- [222] Francesco Caturano, Gaetano Perrone, Simon Pietro Romano, Capturing flags in a dynamically deployed microservices-based heterogeneous environment, in: 2020 Principles, Systems and Applications of IP Telecommunications (IPTComm), IEEE, 2020, <http://dx.doi.org/10.1109/iptcomm50535.2020.9261519>.
- [223] Alessandro Placido Luise, Gaetano Perrone, Claudio Perrotta, Simon Pietro Romano, On-demand deployment and orchestration of cyber ranges in the cloud, in: *CEUR Workshop Proceedings*, Vol. 2940, 2021, pp. 80–91, URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85114928328&partnerID=40&md5=801179997fec8e278fc08ebd96aff0ad>, Cited by: 1.
- [224] Karen A. Robinson, Ian J. Saldanha, Naomi A. Mckoy, Development of a framework to identify research gaps from systematic reviews, *J. Clin. Epidemiol.* (ISSN: 0895-4356) 64 (12) (2011) 1325–1330, <http://dx.doi.org/10.1016/j.jclinepi.2011.06.009>, URL <https://www.sciencedirect.com/science/article/pii/S0895435611002046>.